

CADrawer: Autoregressive CAD Generation from 3D Sketches

Yuanbo Li¹, Gilda Manfredi^{2†}, Henro Kriel^{3†}, Chengye Hao¹, Xianghao Xu¹, Adrien Bousseau³, Daniel Ritchie¹

¹Brown University, USA

²University of Basilicata, Italy

³Centre Inria d'Université Côte d'Azur, France

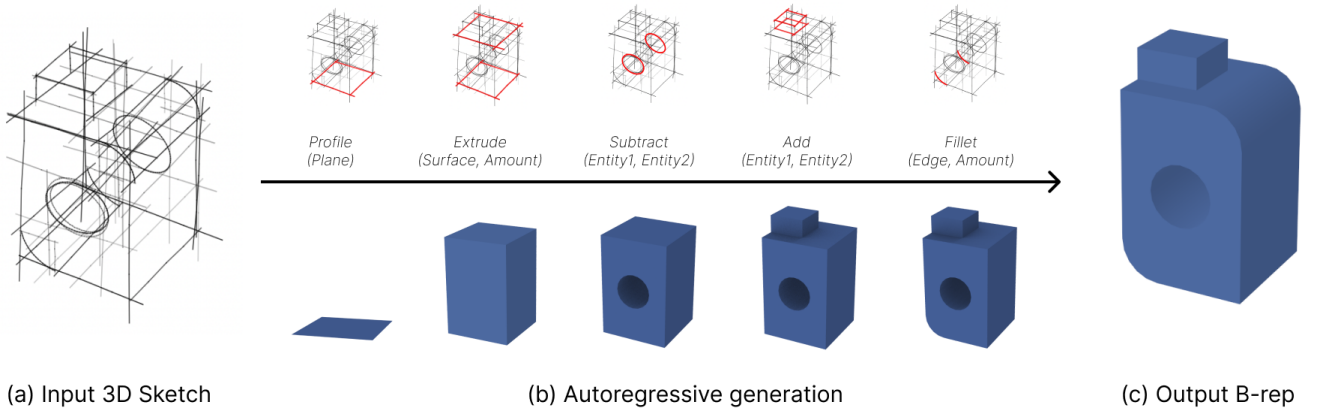


Figure 1: Our system takes as input a 3D sketch, and autoregressively generates a CAD program that produces the intended shape.

Abstract

In professional design workflows, designers often begin by creating sketch drawings before converting them into CAD programs. However, prior work on automatically interpreting these sketches has been limited to simplified inputs and fails to account for construction lines that are ubiquitous in real-world drawings. We present CADrawer, a system that translates 3D sketches into CAD programs using an autoregressive approach, leveraging construction lines as a rich source of information for recovering intermediate CAD operations. At each step, CADrawer predicts the next modeling operation and its parameters based on a graph-based representation of the sketch, which explicitly encodes spatial and temporal relationships between strokes. To improve generation quality, the system maintains multiple candidate programs in parallel, and a learned value function evaluates these partial programs to guide the search toward the most promising candidates. CADrawer is designed as a complement to 3D sketching interfaces, building on existing methods that create 3D sketches. We evaluate our method across several datasets, including those containing dense construction lines and cases without ground-truth B-rep shapes. *This is the authors' version of the work, it is posted here for your personal use. Not for redistribution. The original paper appeared in Computer Graphics Forum, Volume 45 (2026), Number 2.*

CCS Concepts

• *Computing methodologies* → *Shape modeling*;

1. Introduction

Computer-Aided Design (CAD) is a widely adopted standard for creating 3D shapes across various industries. CAD models are typically represented as programs consisting of a sequence of paramet-

ric modeling operations, such as extruding a 2D profile to create a solid block or rounding an edge to create a fillet. The parameters of these operations offer precise control on the dimensions of the geometry produced when executing the programs.

However, creating CAD models requires significant expertise in both planning the sequence of modeling operations and selecting

[†] Equal contribution

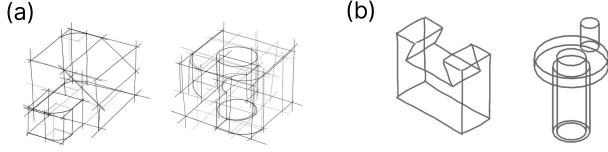


Figure 2: Examples of sketches that our system can process (a), compared to the examples of sketches handled by previous work [LPBM22; LPBM20; SLX*25] (b)

them in feature-rich software interfaces. Meanwhile, sketching offers a quick and flexible way for designers to visualize the 3D shapes they have in mind, and to plan how to construct these shapes in CAD modeling. Prior research [LPBM20; HLMB22] and design educators [Hen12; Sto08] point to strong similarities between the steps designers follow when sketching 3D shapes, and the operations they use to model in CAD software. In this paper, we present a method that exploits these similarities to translate industrial design sketches into CAD programs.

Prior works [LPBM22; LPBM20; SLX*25] have introduced systems that recognize CAD operations from sketches, but these methods are restricted to sketches containing only feature lines, where each stroke directly corresponds to an edge of the final geometry (Figure 2). Relying solely on feature lines makes it difficult to recover complex sequences of additive and subtractive operations, since multiple edits can occur within the same spatial region and the resulting lines may not appear in the finished shape. In contrast, real-world sketches often include construction lines — auxiliary strokes that designers use to outline primary object parts with simple primitives (e.g., cuboids, cylinders) before refining details [GSH*19] additional reference [SIJ*07]. Such lines frequently represent intermediate shapes in the design process or help establish perspective. While construction lines do not appear in the final geometry, many reveal crucial information about the process creating the final shape. Figure 3 gives a typical example of a shape that previous method [LPBM22] fails to recover using only feature lines. In contrast, our method targets sketches with construction lines and takes advantage of this additional information.

The main challenge arises from the fact that construction lines often outnumber feature lines, leading to visual clutter without a direct correspondence to the final shape. As the number of strokes grows, the number of possible loops formed by construction lines also increases, as shown in Figure 4. In addition, designers may omit or repeat strokes, which further increases ambiguity in real-world sketches. The problem we address is to infer CAD operations and their parameters from such noisy and cluttered sketches. Our system operates on 3D sketches, as intersections and planar cycles are more easily detected in 3D than in 2D. These 3D sketches are becoming increasingly accessible through sketch-based modeling interfaces [SKSK09; WB25], sketch reconstruction methods [GHL*20; HGSB22], and VR interfaces [YDSG21]. We demonstrate our approach on 3D sketches that we have created using an existing drawing interface and accompanying reconstruction algorithm [WB25; HGSB22], even though our algorithm could apply to other sources of 3D sketches.

Our key observation is that the geometric relationships between

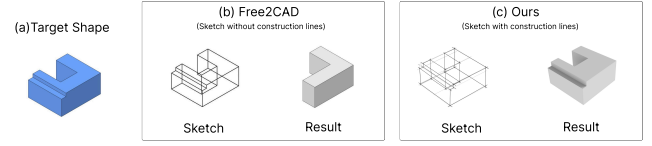


Figure 3: We illustrate a case where Free2CAD [LPBM22] fails to reconstruct the target shape (a) when relying solely on the feature lines shown in (b). Since the strokes corresponding to the subtraction operation are absent, this prior method cannot recover the correct modeling process. In our method, we exploit construction lines for additional information about intermediate structures (c). Our method takes as input feature and construction lines represented as ordered 3D strokes

, enabling successful reconstruction of the intended shape. NOTE: we edited the image to include the with/without construction lines tag for clarity.

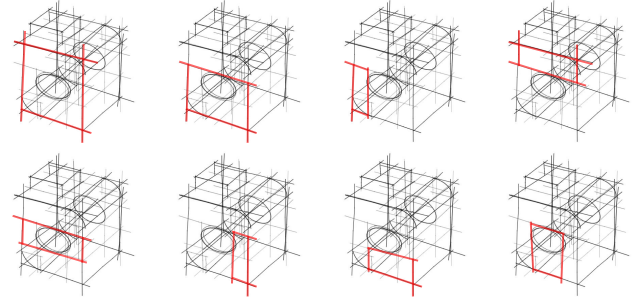


Figure 4: Construction lines can form multiple loops on the same surface, increasing ambiguity and complexity in sketch interpretation.

the strokes convey rich information about the underlying 3D structure. In contrast to prior work that relies on Transformer-based models to discover stroke interactions [LPBM22], we explicitly encode geometric and temporal relationships in a graph where nodes represent sketch entities and edges encode spatial interactions and ordering between these entities. This custom representation allows us to adopt a lightweight graph neural network for analyzing the sketch and predicting the CAD operations. Furthermore, we augment the graph with information from the generated geometry, which provides both spatial and programmatic context.

We adopt an autoregressive approach that predicts a single operation and its corresponding parameters at each step. This sequential formulation allows the model to postpone uncertain decisions and use the progressively built shape to guide more informed and immediate predictions. However, like previous methods, this approach is prone to error accumulation. We maintain a set of candidate programs in parallel and apply Sequential Monte Carlo (SMC) to resample the best candidates. We use a learned value function that evaluates each partial program and concentrates computational resources on the most promising ones during the SMC process.

In summary, our system is the first to tackle the challenge of interpreting sketches that include both feature and construction lines.

It takes as input 3D sketches and autoregressively generates CAD programs that can be executed to produce shapes aligned with the input. We evaluate our system on both synthetic and hand-drawn sketches spanning a range of complexities. Our implementation can be found at: https://github.com/Liyb2002/Loop_Graph

Our main contributions are:

- An autoregressive framework for translating 3D sketches into CAD programs.
- A graph-based representation of 3D sketches that captures geometric relationships between sketch entities.
- A learned value function that evaluates CAD programs by estimating their potential to reproduce the depicted shape.

2. Related Works

Our work builds on two complementary streams of research — sketch-based modeling and CAD program synthesis. We refer to recent surveys for extensive discussions of these two domains [LB25; RGJ*23].

Sketch-based modeling. The field of sketch-based modeling has matured to offer a broad range of interactive and automatic approaches to create 3D shapes from 2D drawings. Optimization-based algorithms tackle this challenge by imposing geometric constraints between lines, such as parallelism and orthogonality [LS96], planarity [LCLT08; YLT13], symmetry [CSMS13; PCV16]. While early methods were limited to polyhedral shapes and clean drawings, later algorithms have been extended to curved objects [XCS*14; SKSK09] additional reference [BBS08; SS14], and sketches with oversketching and construction lines [GHL*20; HGSB22; YDSG21]. Building on this body of work, we assume that our input is a 3D sketch created with these methods. Taking 3D sketches as input facilitates the detection of sketch entities and their spatial relationships, allowing us to focus on recognizing CAD operations from such entities.

We contribute to the family of works that recognize parametric shapes from sketches [HKYM16; NGG*16; LPBM20; LPBM22; PMKB23; SLX*25]. In particular, our method is closest to Free2CAD [LPBM22] that autoregressively identifies groups of strokes that depict CAD operations and derive their parameters. However, both works are limited to simple, clean contour drawings that only contain feature lines that appear in the final shape. In contrast, the design sketches we target contain construction lines, which provide additional information about intermediate CAD operations, but also make the identification process more challenging.

Learning to Recover CAD Programs Our work also relates to the more general goal of reverse engineering CAD models from diverse input, such as voxel grids [SGL*18; TLS*19; LWJ*22], point clouds [WXW18; DIP*18; WXZ21; LOWS23; GLP*22; SLK*20; RDM*24], boundary representations [XPC*21] or others [CF25; WZW*24]. Working on sketches gives us a unique advantage, as the drawing sequence we take as input not only depicts the final shape envisioned by the designer, it also describes how the designer plans to construct it. This additional information helps recov-

ering the ordering of CAD operations, as observed by prior work on sketch-based modeling [LPBM20; LPBM22].

Inspired by prior on deep learning for CAD, we propose to represent 3D sketches with a graph structure that encodes stroke ordering, stroke intersections, and stroke loops. This choice aligns with the inherent nature of CAD boundary representations (B-reps), where graphs naturally capture the relationships between faces, edges, and vertices. Many previous works have proposed their own graph representations tailored to the specific needs of their tasks [XPC*21; CRN*22; WJC*22; JHC*21; JNK*23]. Our representation jointly encodes the 3D sketch and the B-rep generated by executing the CAD program, which enables effective mapping between our input and output while providing spatial context for program generation.

Our approach also builds on ideas from previous works in program synthesis [ENP*19; ERST18; CLS19; TLS*19; KMP*18] that incorporate execution-based feedback into autoregressive generation. Specifically, we adopt an autoregressive approach to capture the sequential nature of CAD programs, where later operations often depend on geometry generated in earlier steps. We extend this paradigm by executing the partial program at each step, comparing the resulting B-rep with the input 3D sketch, and using spatial feedback to guide the next prediction. This execution-feedback loop enables the system to remain aware of construction progress and avoid redundant operations.

3. Approach

Our system takes as input a 3D sketch – a sequence of ordered 3D polylines, each represented by 10 sampled points – and outputs a CAD program that generates the intended 3D shape. We adopt an autoregressive generation process that adds one CAD operation token and its corresponding parameters at each step. Each autoregressive step consists of three actions. First, the system constructs a graph representing the current generation state (Section 4). Next, the policy module predicts the next CAD operation and selects the relevant subset of strokes or loops to determine its parameters (Section 5). Then, the system executes the current program to produce an updated B-rep and compares it against the input 3D sketch to identify which strokes have been explained.

We maintain multiple program samples in parallel. After each step, once all samples have finished execution, a learned value function evaluates their current states, and a resampling step reallocates computational resources to the most promising samples (Section 6).

Similar to previous sketch-to-CAD works [LPBM20; LPBM22; SLX*25] and many other CAD research efforts, our system supports four fundamental CAD operations: `profile`, `extrude`, `fillet`, and `chamfer` (Figure 6). Boolean operations emerge from the `extrude` direction. Extruding outward `add` material, while extruding inward `subtract` material.

4. Graph Representation

At each autoregressive step t , we construct a heterogeneous graph $G_t = (V, E)$ where nodes V represent sketch entities, and edges E encode spatial relationships and sequential orderings between these

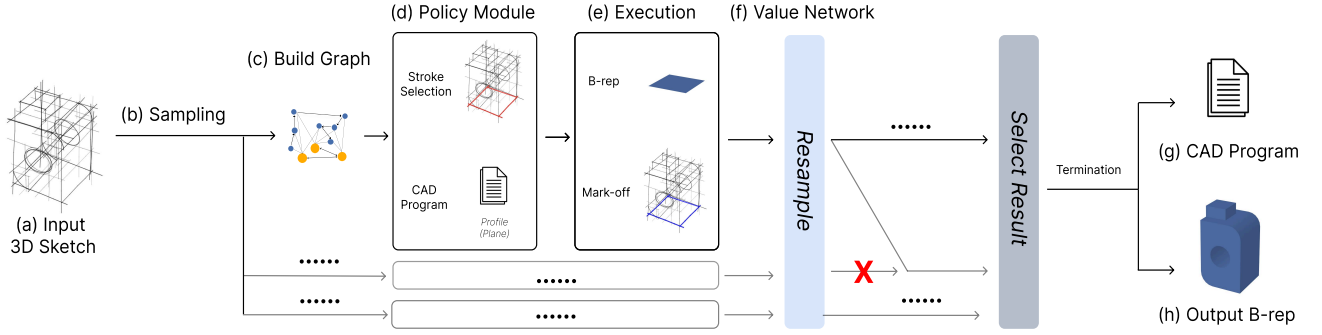


Figure 5: Our system takes as input (a) a 3D sketch and performs autoregressive generation to produce (g) a CAD program, and (h) the resulting B-rep shape by executing the program. We create multiple samples that run in parallel, which are resampled after each step to maintain diversity and guide the generation progress. At each autoregressive step, we first build a graph (c) representing the current state of the reconstruction (Section 4), and then the policy module (d) predicts a CAD operation and identifies the strokes used to derive its parameters (Section 5). The current program is then executed and compared with the input 3D sketch to mark off the strokes that are already represented in the current program (e). This feedback is used as input for the next step. After each step, the value function (f) estimates the likelihood of success for the current program state, allowing us to focus the search on more promising samples (Section 6).

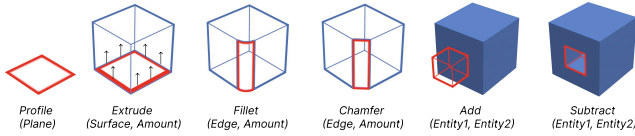


Figure 6: Our system supports six operations: *profile*, *extrude*, *fillet*, *chamfer*, *add*, and *subtract*.

entities.(Figure 7a,b). To capture the program state, we execute the partially generated CAD program to produce a B-rep and compare it against the input 3D sketch to identify which strokes have already been explained (Figure 7c). This information is encoded as a binary usage flag on the corresponding nodes. This comparison provides spatial grounding, as it is difficult for neural networks to perform spatial reasoning solely from symbolic program tokens. The resulting unified graph is in one-to-one correspondence with the evolving CAD program, ensuring that each program state has a unique graph representation. This graph serves as input to both the policy module and the value network, providing information from both the sketch and the CAD program.

Prior work such as [YZF*21] represents sketches as graphs where nodes correspond to sampled points and edges to stroke segments. However, this approach captures only local geometry and struggles with more complex sketch structures. In contrast, Free2CAD [LPBM22] models sketches as sequences using a Transformer-based architecture to capture temporal order of strokes, but neglects spatial relationships and incurs substantial computational costs (9 days of training reported). Concurrently to our work, Sketch2Seq [SLX*25] is based on a graph structure that encodes strokes as nodes and local and distant spatial relationships as edges, but it ignores stroke ordering and larger entities such as loops formed by successive strokes.

Our method combines the strengths of these approaches: we en-

code both sequential and spatial relationships in a unified graph structure using heterogeneous edge types. This allows for efficient processing with a lightweight graph neural network that can be trained within a few hours. Furthermore, our graph includes two types of nodes: *stroke nodes* and *loop nodes*. Loop nodes represent coplanar, closed groups of strokes that typically define profile regions for planar operations. These nodes ensure that the profile detection module can consistently identify closed, complete sketch planes. Another challenge is the *ambiguity of stroke roles*, where the purpose of a stroke may only become clear after earlier parts of the sketch are interpreted. Our graph representation addresses this by allowing each stroke to reason about its spatial and temporal neighbors and the usage status of those neighbors. We provide pseudocode for the graph construction pipeline in Appendix A.

4.1. Graph Nodes

The input 3D sketch is represented as a set of polylines, with each polyline sampled at 10 points. For each stroke, we fit a parametric function (e.g start and end points of a line) based on its geometry, including: straight lines, circular arcs, full circles, ellipses, and free-form curves. Each stroke node in our graph encodes the corresponding parametric function, the stroke’s opacity, its type, and a binary label indicating whether it is used in the Brep when executing the current program. In contrast, each loop node contains only a binary indicator for B-rep usage. We provide additional details on the node feature representations in Appendix B.

4.2. Graph Edges

The graph edges capture both the spatial relationships between nodes and the temporal order of stroke execution. Stroke-order edges are directed, reflecting the sequence in which strokes are drawn, whereas all other edges are undirected and defined solely by geometric relations. To assess the contribution of each edge type,

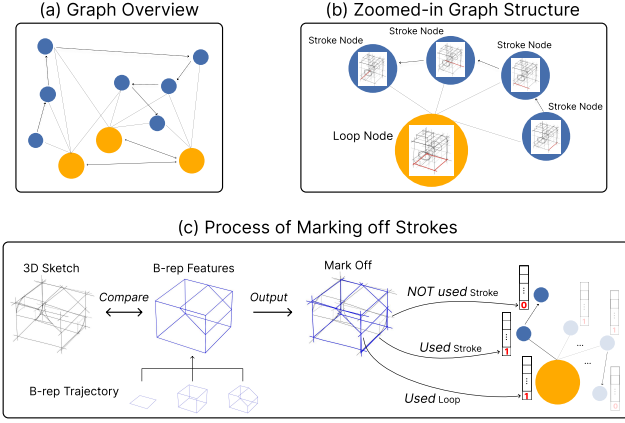


Figure 7: An overview of the graph (a), and a zoomed-in view (b). Panel (c) shows the process of marking off strokes. We execute the CAD program incrementally to produce all intermediate shapes generated throughout the process. This is because certain edge features, especially those involved in *subtracts*, may not appear in the final shape. The resulting mark-off (in blue) indicates which strokes have been explained by the current program.

we perform an ablation study in Section 8.5. The edge categories are as follows:

- **Stroke-to-Stroke Edges:** Capture intersection between strokes in the 3D sketch.
- **Loop-to-Loop Edges:** Capture intersection, containment (which loop contains which), and perpendicular relationships between loops.
- **Stroke-to-Loop Edges:** Indicate which strokes constitute a particular loop.
- **Stroke-order Edges:** Capture the order in which strokes were drawn.

The detailed algorithm for building the graph edges can be found at Appendix C.

5. Policy Module

Our system autoregressively generates a CAD program $\mathcal{P} = \{p_t\}_{t=1}^T$, where each $p_t = (o_t, \theta_t)$ denotes a CAD operation o_t and its associated parameters θ_t . At each timestep t , the policy module takes as input the graph constructed in Section 4 and performs three tasks: (1) predicting the next CAD operation o_t (Section 5.2); (2) selecting the relevant strokes from $\mathcal{S}$ (Section 5.2); and (3) inferring the operation parameters θ_t based on the selected strokes (Section 5.3). Example programs can be found in Appendix D.

5.1. Graph Encoder

We use a shared Graph Convolutional Network (GCN) encoder to compute node embeddings from the input graph G_t . These embeddings are then fed into task-specific decoders for different tasks. We provide detailed architecture of the network in Appendix E.

5.2. Task-Specific Decoders

We design different decoders tailored to different tasks, each following a specific pipeline (see Figure 8), and train them separately. For each decoder, we initialize a separate graph encoder and jointly train the encoder-decoder pair on the corresponding target task.

5.2.1. Operation prediction.

To predict the next CAD operation token (Figure 8a), we perform cross-attention between the program embedding (as query) and the graph embeddings (as key and value), thereby annotating the program with geometric context. We then apply self-attention over the annotated program embedding (the [CLS] token) to aggregate information and produce the next program token. Our loss function is the standard cross-entropy loss, which penalizes the model when it assigns low probability to the correct operation token, in which $\hat{y}_{op}$ denotes the predicted probability distribution over the operation tokens and k denotes the index of the ground-truth operation token:

$$\mathcal{L}_{op} = -\log \hat{y}_{op,k}, \quad (1)$$

5.2.2. Stroke and Loop Feature Selection.

For operations that require geometric input, such as selecting a loop for *Profile*, strokes for *Extrusion*, or strokes for *Fillet/Chamfer*, we perform binary classification over the relevant nodes (Figure 8b,c,d). For each node v , we compute a selection probability by minimizing the following focal loss:

$$\mathcal{L}_{stroke} = - \sum_{v \in \mathcal{V}_{stroke}} \alpha (1 - \hat{p}_v)^\gamma y_v \log \hat{p}_v, \quad (2)$$

where $y_v \in \{0, 1\}$ indicates whether stroke node v is selected, and $\hat{p}_v \in (0, 1)$ is the model-predicted probability that v is selected. We use the focal loss [LGG*17] with $\gamma = 1.5$ to down-weight easy examples via the factor $(1 - \hat{p}_v)^\gamma$, mitigating the strong class imbalance since only a small fraction of nodes are selected at each step. We always set $\alpha = 1.0$ for uniform class weighting.

The loss function in Eq. (2) serves as the common objective for all geometric selection tasks. The specific pipeline for constructing the candidate set of graph nodes, however, differs by task, as described below:

- **Profile selection (b):** An MLP is applied to the loop embeddings, followed by binary classification. The loop with the highest probability is selected.
- **Extrusion (c):** We identify the extrusion face using a two-round encoding-decoding process: the first round predicts extruded strokes from the flagged profile loop, and the second round uses these flagged strokes to determine the resulting extrusion face. We provide more details of this approach in Appendix G.
- **Fillet and chamfer selection (d):** An MLP is applied directly to the stroke embeddings, followed by binary classification. The contributing strokes are then selected.

5.3. Finding Operation Parameters

Given the strokes (or loops) selected for each operation, we extract continuous values required to execute that operation. A major

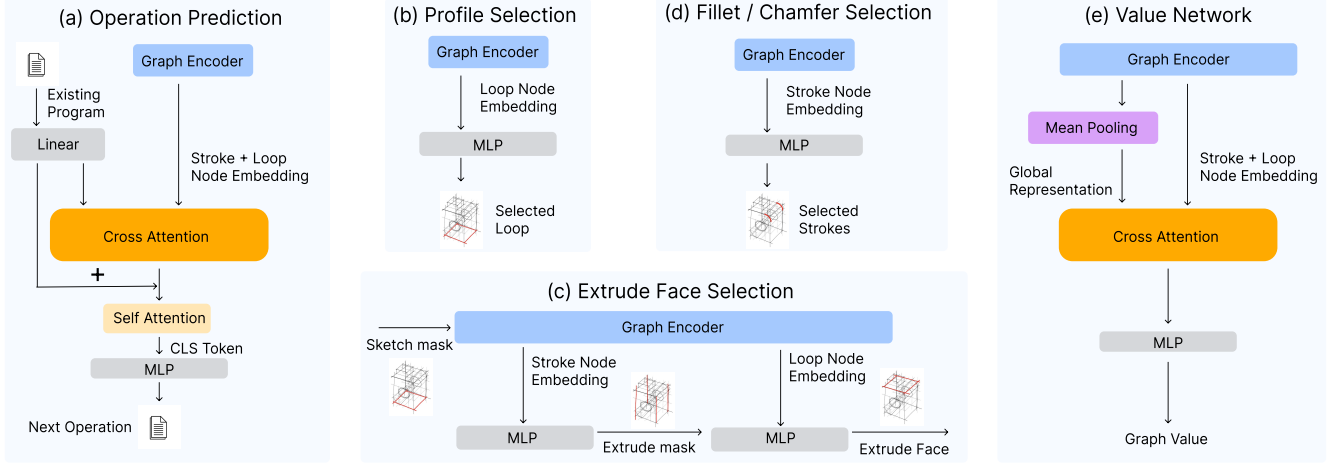


Figure 8: Overview of the decoder architecture. Each submodule is responsible for a specific task: (a) operation prediction, (b) profile selection, (c) extrude face selection, (d) fillet/chamfer selection, and (e) value network.

challenge is that the input strokes are sketches that are inherently imprecise, making it difficult to recover exact parameter values directly. To address this, we employ a set of geometric algorithms to infer the parameters, as detailed in Appendix F.

6. SMC Based Program Sampling

Performing the entire autoregressive generation process in a single shot is challenging. First, errors can accumulate across steps, compounding over time. Second, 3D sketches are often ambiguous so that multiple valid interpretations may exist, and later decisions may depend on earlier ones. To capture this uncertainty and maintain a diverse set of plausible solutions, we adopt a Sequential Monte Carlo (SMC) framework that maintains a set of samples CAD programs, referred to as *particles*.

All particles are initialized from the same state: the empty program. At each timestep t , each particle samples its next step from the policy module, which involves predicting the next operation token and selecting the corresponding strokes. This procedure defines the prior distribution:

$$p(x_{0:t}^{(i)}) = \prod_{k=1}^t p(x_k^{(i)} | x_{0:k-1}^{(i)}),$$

where $x_k^{(i)}$ denotes the program step chosen at time k by particle i .

SMC then approximates the posterior distribution $p(x_{0:t} | y)$, where y is input graph G_t . As directly computing this posterior is intractable, SMC resamples the particles based on a learned value function $V(x_{0:t})$ (Section 6.1). This resampling helps recover from early mistakes and maintain diversity among plausible particles, which is particularly important for complicated sketches. We show an example of this process in Figure 9, and we describe the resampling algorithm in details in Appendix H.

6.1. Value Function

We need a scoring function that evaluates how well a candidate CAD program matches the target sketch. Since different execution

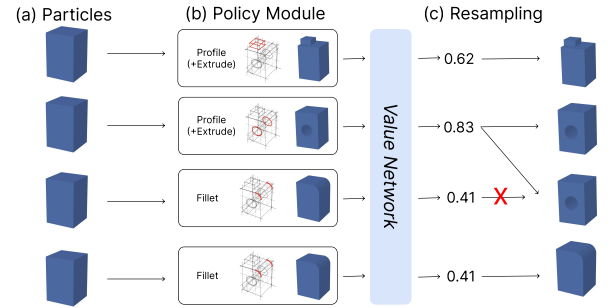


Figure 9: We present an example of resampling using SMC. After all particles pass through the policy module, the value network assigns each of them a score. The SMC then resamples based on these scores, shifting the distribution toward particles with higher likelihood.

orders of CAD operations can produce the same final B-rep, this value function must be order-invariant.

Previous works on CAD generation often evaluate their results by computing the Chamfer Distance between the generated B-rep and inputs such as voxels [UCS*22; KSA23], point clouds [GLP*22; ZHFL23; LCP*24], or meshes [GXL13]. In contrast, directly comparing our generated B-rep with the input 3D sketch is not meaningful. Such a comparison only reveals which strokes have been explained by the program. But many construction lines are not intended to appear in the final shape, and the input sketch itself is sparse.

Instead, we evaluate the generation process by computing the Chamfer distance between the generated B-rep and the ground-truth B-rep. However, during inference, the ground-truth shape is not available, making direct computation infeasible. To address this, we train a neural network that takes the current graph G_t as input and learns to predict a proxy for the Chamfer distance. This

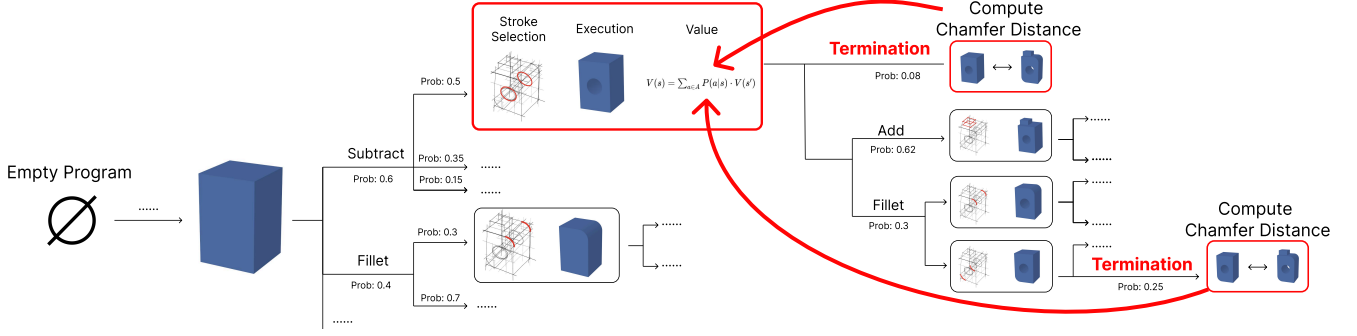


Figure 10: We build a tree from a partial CAD program by simulating future actions. Each branch represents a possible choice by the policy module. Non-terminal states’ values are based on their child nodes weighted by probabilities. Terminated states are evaluated using Chamfer distance to the ground truth shape.

learned value function enables geometry-aware scoring of partial CAD programs without the ground truth during generation.

6.1.1. Immediate Value Estimation

A straightforward approach is to train our neural network to predict the Chamfer distance S_f of the current B-rep. However, as Chamfer distance is correlated with the volume of the shapes, operations that create larger volumes (e.g., `extrude`, which produces a solid block) might have greater impacts on the Chamfer distance than operations that modify smaller features (e.g., `fillet`, which rounds edges). In our experiments, we observe that the SMC sampling process with this immediate value estimation tend to favor samples that prioritized `extrude` operations, leading to a greedy search behavior.

6.1.2. Expected Value Estimation

A more principled way to evaluate a partial CAD program is by estimating the quality of its expected final output. Inspired by prior works such as AlphaGo [SHM*16; SSS*17], we construct a search tree that explores possible future executions from the current program state (Figure 10). The value of a partial program is then computed by aggregating the values of all possible completions, weighted by their probabilities. We define the value of this partial programs as:

$$V(s) = \sum_{a \in A} P(a|s) V(s')$$

where:

- $V(s)$ is the value of the current state.
- A is the set of possible operations from s .
- $P(a|s)$ is the probability of writing program a from state s .
- s' is the next state obtained by applying a to s .
- $V(s')$ is the value of the next state, or the Chamfer distance if it is an termination state.

However, constructing a complete tree that explores all possible executions of the system is computationally infeasible. We approximate this process using Monte Carlo Tree Search (MCTS), which focuses exploration on high-impact branches. Implementation details of our MCTS algorithm are provided in Appendix I.

To train the value network, we use the search trees generated by our MCTS procedure to construct a dataset that provides estimated values for program states at various stages of execution. We adopt the same graph encoder (detailed in Appendix E) to produce node embeddings, then passed through a value decoder (Figure 8) to predict a scalar value representing the estimated quality of the current state. We train the value network using a contrastive loss that encourages higher scores for better programs:

$$\max(0, m - y_{ij}(S_i - S_j)),$$

where (i, j) denotes a sampled pair of programs from distribution $\mathcal{D}$, S_i and S_j are their predicted scores, $y_{ij} \in \{1, -1\}$ indicates whether program i is better than program j , and $m = 0.2$ is the margin.

7. Implementation

7.1. Dataset

There exist multiple datasets for CAD, such as DeepCAD [WXZ21], the ABC dataset [KMJ*19], and the Fusion360 Gallery [WPL*21]. However, none of these datasets provide sketches suitable for training our model. In contrast, the OpenSketch dataset [GSH*19] offers paired sketches and 3D meshes, but it contains only 12 shapes, these shapes require a broader set of CAD operations than the four supported by our current system, and the dataset does not provide the sequence of operations that produced each shape.

We develop a novel method to generate noisy synthetic 3D sketches that imitate human sketches (detailed in Appendix J) and prepare two datasets using it. The first dataset (referred as Dataset A in later discussions (Figure 12 and Figure 14), builds on the CAD data introduced by [HLMB22] and consists of 1361 CAD program and 3D sketch pairs (with construction lines) and includes `profile`, `extrude`, and `fillet` operations. Each program contains exactly 8 operations, and the resulting sketches have an average of 78.6 strokes, with a minimum of 35 and a maximum of 143 strokes. To increase diversity and complexity, we procedurally generate a

second dataset (referred as Dataset B in later discussions) comprising 4000 CAD program and 3D sketch pairs (Figure 13 and Figure 15), covering all four basic operations: `profile`, `extrude`, `fillet`, and `chamfer`. Program lengths range from 3 to 15 operations, with an average of 9.2. The resulting sketches vary from 17 strokes at the simplest end to 307 strokes at the most complex, with an average of 122.3 strokes.

Both datasets are divided into 80% for training and 20% for validation. They feature diverse designs (exampled in Figure 14, 15) and differs in program length, program patterns, spatial relationships between strokes, as well as in how feature lines and construction lines are drawn. We train on these datasets jointly to highlight generality. In Section 8, we present results from training both separately and jointly (by randomly merging them into a single combined dataset), demonstrating our system’s ability to generalize across a wide range of sketching styles.

7.2. Network Training and Inference

We implement our neural networks in PyTorch Geometric and will release the code upon acceptance. Training is performed on an NVIDIA GeForce RTX 4090 GPU. The policy networks (including four stroke/loop entity selection tasks and the operation prediction network) are trained in approximately 2 hours in total. The value network takes ~ 10 hours. At inference time, our system generates a CAD program (9 operations) in ~ 30 seconds using 30 parallel particles in the SMC framework.

8. Results and Evaluation

8.1. Baseline Method : Order Based Reconstruction

We implemented a baseline algorithm that processes strokes in the order they were drawn. In this approach, strokes are sequentially added, and whenever they form a closed loop, the algorithm groups them into a sketch loop. When such loops correspond to higher-level entities (e.g., a cuboid), the algorithm generates the corresponding sketch and extrude operations to construct the intended geometry.

However, this approach faces two major challenges. First, artists often draw in inconsistent order, sometimes revisiting earlier parts of the sketch. Second, sketches frequently include construction lines, which can form loops with feature lines. This algorithm often mistakenly interpret these as valid sketches, resulting in errors. To evaluate this method, we selected 100 short programs from Dataset B (each containing only six operations). The baseline succeeded in generating only 1 out of 100 shapes, clearly illustrating its limitations.

8.2. Baseline Method : Stroke Filtering as Preprocessing

Another baseline method we consider is a two-stage pipeline. The first stage selects strokes that either appear in the final shape or in intermediate shapes, since these strokes can help generate the entire CAD generation process. Such strokes include both feature lines and a subset of construction lines. Our objective is to use only these selected strokes to predict the CAD program, thereby reducing the

burden on the network. Specifically, we train a network to perform binary classification of strokes, separating those that are ever used in the shape’s generation history (i.e. present in intermediate shapes or the final shape) from those that are not (i.e. construction lines used solely for perspective correction). This classifier adopts the same graph encoder as our main pipeline to compute node embeddings, followed by a multilayer perceptron (MLP) that operates on the stroke nodes.

We evaluated this approach on 500 shapes sampled from Dataset B. The preprocessing network achieved an accuracy of 86.2% in distinguishing between strokes that are used and those that are not used during the construction process. However, only 173/500 examples retained all the lines required to fully generate the program. For the remaining 327/500 examples, recovering the correct program was difficult regardless of the generation algorithm. This stroke pre-processing does not work well because it is inherently challenging to determine which construction lines are essential for the generation process in a single-shot prediction. In contrast, our method (proposed in this work) addresses this challenge through an autoregressive formulation, where later predictions can build on earlier ones, making it easier to capture the necessary lines for program recovery. We provide example results of predicting lines that are used in the shape’s generation history in Figure 11.

8.3. Overall Performance

We train our network on the two datasets both separately and jointly. Joint training on Dataset A and Dataset B enables broader generalization, but it also introduces challenges due to stylistic inconsistencies between the datasets. For example, Dataset B often uses diagonal lines to denote profile planes, whereas Dataset A does not (Figure 14, Figure 15). Such differences can confuse the network, since identical operations are represented with different visual cues. Nevertheless, our system remains capable of making valid predictions by reasoning about underlying spatial relationships rather than relying solely on dataset-specific patterns. This indicates that the model learns to infer higher-level geometric intent, contributing to its robustness.

To assess shape quality, we compute the Chamfer distance between the generated shape and the ground-truth shape in the validation set, using 300 uniformly sampled surface points. A generation is considered successful if the Chamfer distance is less than 1% of the bounding box diagonal of the ground truth shape. We also present several failure cases and their underlying causes in Figure 16.

We observe that the value function often struggles to distinguish fine-grained shape differences, particularly those involving small features such as `fillet` or `chamfer` operations. For instance, on Dataset B with the MCTS value function, the top-1 accuracy is 83.5%, compared to 91.0% for top-3. This gap reflects cases where multiple candidates exhibit similar Chamfer distances at the end of the generation process, making top- k outputs more reliable for user selection. More comparisons can be found in Table ?? . To mitigate this limitation, our system returns the top-3 predicted shapes, ranked by the values given by the value function, and allows users to select their preferred result.

Table 1: Top-1 vs. Top-3 success rate (%) (Chamfer distance < 1% of the ground-truth bounding-box diagonal) using the MCTS-based value function.

	Dataset A	Dataset B	Joint (A + B)
Top-1	76.5%	83.5%	70.0%
Top-3	82.0%	91.0%	82.5%

Table 2: Top-3 success rate (%) (Chamfer distance < 1% of the ground-truth bounding-box diagonal) with different sampling strategies.

Sampling / Value Function	Dataset A	Dataset B	Joint (A + B)
No Sampling / No Value Function	59.0%	67.0%	48.0%
Uniform Sampling	60.0%	67.0%	47.5%
Based on number of strokes used	67.0%	72.5%	59.5%
Immediate Value Function	82.0%	89.0%	80.0%
MCTS-based Value Function	82.0%	91.0%	82.5%

In Table ??, we compare the effectiveness of three sampling strategies: (1) a baseline without resampling, (2) Uniform Sampling, (3) Sampling solely based on the number of strokes used at each step, (4) SMC sampling with resampling based on an immediate value function (Section 6.1.1), and (5) SMC sampling guided by a value function trained to estimate the expected final outcome (Section 6.1.2).

Comparing (1) (2) and (3) in Table ??, we observe that uniform sampling does not lead to a substantial improvement compared to no sampling, as both strategies effectively correspond to either no resampling or random resampling of particles. In contrast, when we incorporate the number of strokes used as a sampling value—encouraging particles that utilize more strokes to be sampled more frequently—we observe marginally improved results. This is because simply favoring particles with more strokes is not always beneficial: it can bias the search toward selecting planes with many strokes or extrude operations that generate additional strokes, rather than toward semantically correct constructions.

Comparing settings (4) and (5) in Table ??, we find that the value function trained on expected final values does not yield improvement over the immediate value function on Dataset A, while providing a more noticeable benefit on Dataset B. This is likely because all programs in Dataset A follow a fixed operation sequence. As a result, greedy strategies that prioritize high impact operations like extrusion do not lead to incorrect programs. Though they still provide a substantial improvement over not using sampling at all (1).

8.4. Accuracy on Individual Tasks

We further assess the accuracy of individual modules (Table 3), covering operation prediction and stroke selection for `profile`, `extrude`, `fillet`, and `chamfer`. Chamfer accuracy is not reported for Dataset A, since it contains no chamfer operations, and

Table 3: Accuracy (%) for operation prediction and corresponding strokes (or loops) selection across different dataset setups.

Task Type	Dataset A	Dataset B	Joint (A + B)
Profile	88.7%	94.2%	82.2%
Extrude	94.2%	97.4%	93.3%
Fillet	89.6%	99.6%	94.5%
Chamfer	/	82.7%	/
Next Operation	99.7%	89.9%	92.1%

NOTE: This table extends into the right column because of the edits tag. In the final version, once the tag is removed, the formatting will display correctly.

Table 4: Ablation study of graph design. We report average accuracy for `Profile`, `extrude`, and `fillet` selection tasks.

Edge Type Removed	Profile	Extrude	Fillet
Full Graph	88.7	94.2	89.6
No Stroke-intersect-Stroke	86.7%	65.1%	82.1%
No Loop-perpendicular-Loop	81.1%	84.4%	87.7%
No Loop-contains-Loop	69.3%	93.6%	89.2%
No Stroke-to-Loop	82.3%	13.5%	85.6%
No Stroke Order	67.2%	92.8%	70.4%
No Loop Nodes	45.6%	/	/

is also omitted for joint training, as the results are identical to those of Dataset B. A prediction is considered correct only if all corresponding strokes (or loops) are selected. The task is considered a failure if any extra strokes or loops are selected.

8.5. Ablation Study: Graph Design

We examine our graph design by removing different graph edge types and record the network’s performance on `profile`, `extrude` and `fillet` stroke selections on Dataset A. We show in Table 4, that removing any of these graph edges would lead to a decrease in certain tasks. Additionally, we experiment with a graph that contains only stroke nodes. In this setting, the `profile` prediction is reformulated as identifying all strokes that form the profile region. The result of this variant is shown in the last row of the table.

8.6. Impact of Program Length and Sampling Budget

Our system’s performance declines as the length of the target CAD program increases. Also larger number of particles during the SMC sampling process may improve results. We quantify this relationship using Dataset B (which has varying program length) in table 5.

Our system performance degrades significantly for programs longer than 10 steps, and especially beyond 12. These failure cases often involve missing smaller geometric features, such as `fillets` or `chamfers` (Figure 16). This degradation is likely due to several factors. First, longer programs correspond to sketches with

Table 5: Top-3 results success rate (%) (Chamfer distance is less than 1% of the bounding box diagonal) on Dataset B across different program lengths and numbers of SMC particles.

Particles	< 5 Step	5–7 Step	8–10 Step	11–15 Step
30 Particles	99.2%	94.4%	82.9%	38.5%
50 Particles	99.2%	95.1%	83.9%	48.0%
100 Particles	99.2%	95.1%	87.4%	52.1%

more strokes, which inherently increases difficulty of the generation process. Second, autoregressive models are more prone to errors as sequence length increases. Third, the value estimation function performs less reliably on complicated densely sketches, which makes it hard to identify the 3 most promising final outputs.

8.7. Comparison with Free2CAD

We demonstrate that incorporating construction lines enables our method to reconstruct shapes that previous approaches, such as Free2CAD [LPBM22], fail to capture. The limitation arises because relying solely on feature lines makes it difficult to recover the complex sequence of additive and subtractive operations. Multiple edits may occur in the same spatial region and their traces are often absent in the final geometry.

In Figure 17, we highlight six examples taken from the Free2CAD supplemental material where the system was unable to generate the correct shapes. (Most results provided in this supplemental material succeed, leaving very limited failed examples to compare with). In these examples, Free2CAD takes input sketches with only feature lines, which provides too little information to reconstruct the correct shape. To enrich the information, we redrew the sketches with construction lines before processing them with our method. The inclusion of construction lines provides additional cues about intermediate structures in the modeling process, allowing our approach to accurately interpret them and produce final shapes that more closely match the sketch’s intent. We stress that although our method succeeds on these examples, it operates on sketches that differ from the original Free2CAD inputs. We also emphasize that our comparison focuses on selected failure cases from Free2CAD and does not reflect its overall performance or correctness.

Another possible comparison is to evaluate both systems on sketches that include construction lines. However, as the setup of Free2CAD differs from ours and is designed to use all strokes provided in the input, it cannot function meaningfully on sketches that contain many construction lines unused in the final shape.

Finally, we evaluate our system on the same sketches containing only feature lines (Figure 18). Under this setting, our method also fails in four out of six cases. This further highlights the importance of construction lines in providing critical information for the reconstruction process.

8.8. Evaluating on Synthetic 2D Sketches

We qualitatively evaluate our method on synthetic 2D sketch drawings that are lifted to 3D to simulate noisy 3D sketches. Specifically, we first sample a subset of examples from CAD2Sketch [HLMB22], which generates 2D sketches from 3D shapes. We then uplift these sketches into 3D space using a symmetry-based algorithm [HGSB22]. As shown in Figure 19, our method successfully reconstructs the intended shapes.

8.9. User Study: Creating 3D Shape from 2D Sketches

We further evaluated our method on real-world 2D drawings. Specifically, we invited three students with limited prior CAD design experience and one student designer proficient in CAD design. We asked each participant to draw freehand sketches in 2D using a drawing interface that lifts 2D sketches into 3D [WB25; HGSB22] (see the interface in Appendix K). The resulting 3D sketches were then processed with CADrawer to generate 3D B-rep shapes. Each participant first received a brief 15-minute tutorial on the UI system (Appendix K) and on perspective drawing. They were then asked to produce three sketches of their choice in 2D space, which the system automatically uplifted into 3D sketches. We did not provide target shapes, so participants could draw freely. We encouraged users to draw more complex shapes. However, maintaining correct perspective during sketching is difficult, and inaccuracies in perspective can adversely affect the performance of the 3D lifting algorithm for downstream tasks. While the participants exhibited diverse sketching habits, most of them used construction lines, consistent with our assumption (further discussed in Appendix L).

On average, participants spent about 21 minutes completing all three sketches. Students with limited prior CAD design experience found perspective drawing increasingly difficult as the sketches grew more complex, whereas the proficient student designer found our UI more intuitive and convenient. We then applied CADrawer to translate these 3D sketches into CAD programs, with the results presented in Figure 20.

We used a 5-point scale (1 = very unsatisfied/very different, 5 = very satisfied/highly similar) to evaluate participant feedback. Overall, participants reported a high level of satisfaction with both the sketching process and the automatic 3D lifting. The average similarity score was 4.6/5, indicating that the generated shapes were generally considered close to the original sketches. The system also received an average ease-of-use rating of 4.2/5. All participants with limited CAD experience agreed that it made creating 3D shapes easier than working directly with CAD software. In contrast, the proficient student designer found direct modeling in CAD software easier and more intuitive.

8.10. User Study: Expert Manual Shape Reconstruction

We also conducted a second user study to directly compare human experts in reconstructing 3D shapes from sketch drawing with the automated generation process of CADrawer. We invited three student designers from a prestigious design school, each proficient in CAD software and experienced in manual modeling workflows. In this study, participants were provided with six 3D sketches

and asked to reconstruct the corresponding B-rep shapes manually, without the assistance of our system.

During the process, we observed that participants often struggled with sketches that involved complex modeling steps, particularly those requiring multiple subtraction operations. Overlapping strokes frequently created visual ambiguities, making it difficult to determine the intended sequence of operations. Participants also encountered challenges in accurately interpreting perspective from the sketches, whereas CADrawer automatically extracts precise geometric parameters from strokes.

At the same time, human designers demonstrated strong contextual reasoning and an ability to infer design intent beyond what was explicitly drawn. This often allowed them to avoid certain mistakes made by our system, such as misinterpreting partially drawn or ambiguous strokes. Notably, they could still infer correct parameters even when stroke values extended beyond the thresholds used by our algorithm.

We present a side-by-side comparison of the manually created shapes and the results generated by our system in Figure 21. This comparison illustrates the complementary strengths of expert human reasoning and automated CAD generation.

9. Conclusion

We introduce CADrawer, a new framework for generating CAD programs from 3D sketches. To the best of our knowledge, we are the first to leverage sketch construction lines as additional information for recovering intermediate CAD operations, which allows us to successfully manage complex sketches that challenged previous approaches. But construction lines bring additional clutter and ambiguity, which we handle with a combination of autoregressive prediction and Sequential Monte Carlo exploration.

A key challenge that remains is the lack of large-scale datasets of real-world sketch drawings and their corresponding B-rep programs. Acquiring such data is difficult due to the manual effort required. This limitation constrains the diversity of training samples and hinders the model's ability to generalize across varying sketching styles and modeling workflows. This limitation, shared by Free2CAD and prior methods, highlights an important direction for future work.

One promising direction is to adopt a bootstrapped program synthesis strategy, as explored in [JWR22; EWN*21; JGMR23], where two networks are jointly trained to synthesize programs from sketches and generate human-like sketches from programs at the same time. This approach enables training an initial inference model on a small dataset, executing it to generate synthetic sketch-program pairs, and iteratively refining both the model and dataset via self-supervised learning.

Acknowledgments

This work was supported by the ANR-NSF grant NaturalCAD (ANR-23-CE94-0003, NSF #2315354). Part of this work was conducted while Gilda Manfredi visited Inria for 6 months.

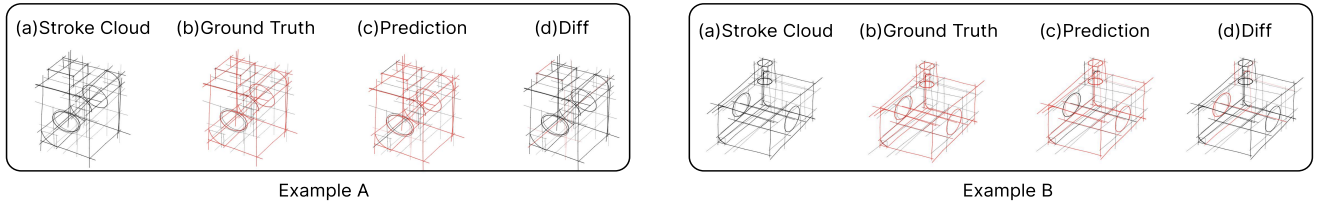


Figure 11: Stroke filtering as a preprocessing step. We show two sets of results where strokes are classified as either used in the generation history or not. In both examples, some essential strokes are excluded, making the correct reconstructing infeasible.

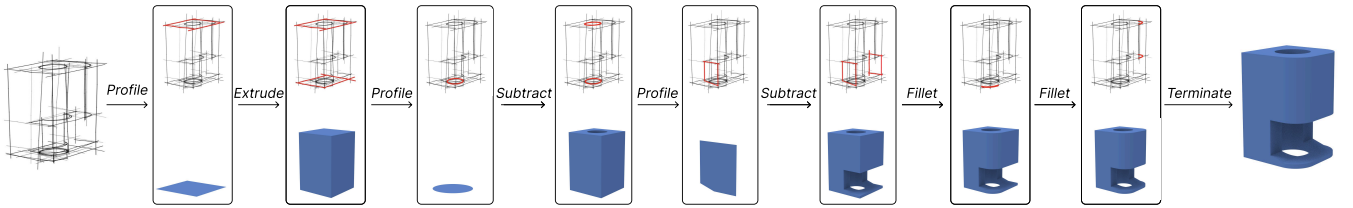


Figure 12: We show the entire process of generating a CAD program from Dataset A. For each step, the selected strokes (highlighted in red) are shown at the top of the box, while the generated B-rep is shown at the bottom.

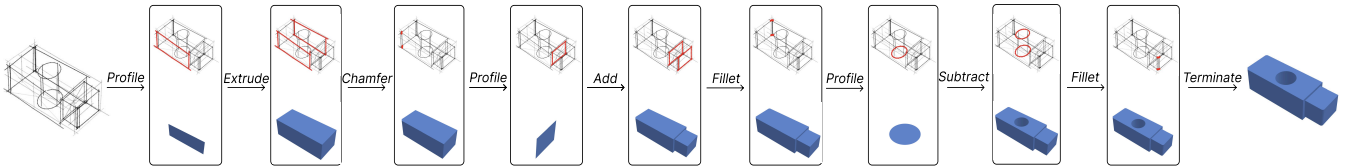


Figure 13: We show the entire process of generating a CAD program from Dataset B. For each step, the selected strokes (highlighted in red) are shown at the top of the box, while the generated B-rep is shown at the bottom.

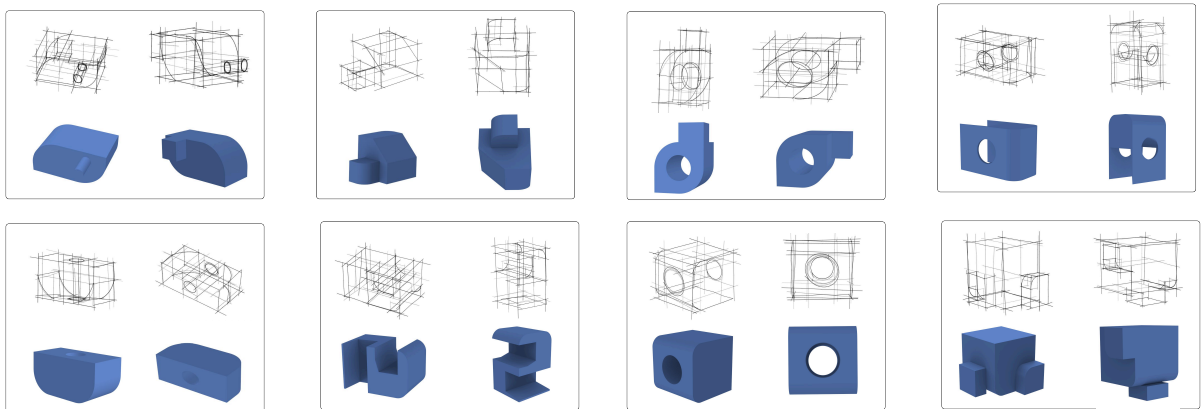


Figure 14: We show eight results of CAD program generation from Dataset A. Each box contains one result, with the shape shown from two different perspectives.

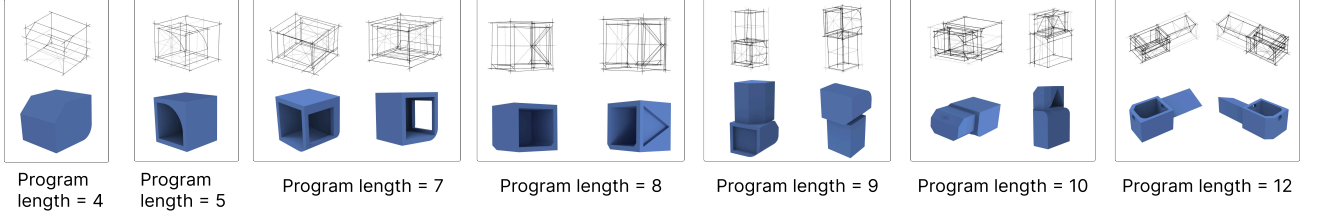


Figure 15: We present seven results of CAD program generation from Dataset B with varying program length. Each box contains one result, with the shape shown from one or two different perspectives.

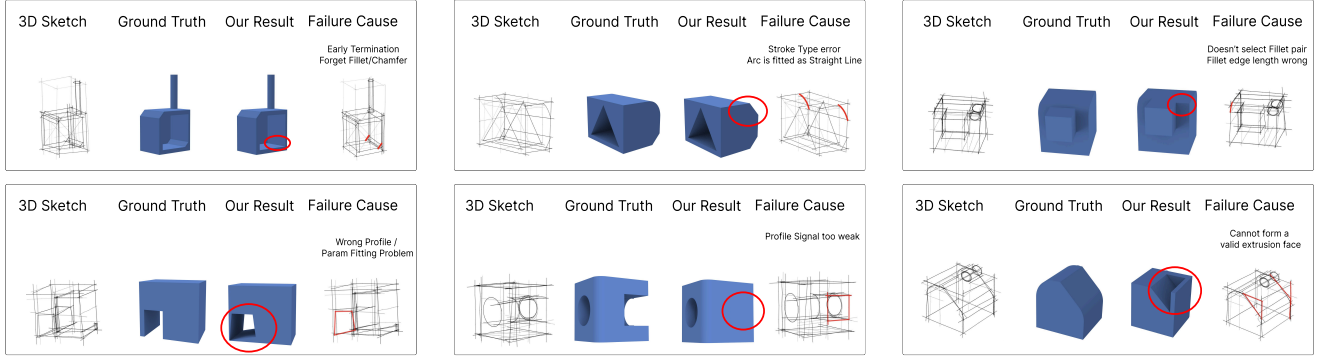


Figure 16: We present six failure cases. In each box, we show the input 3D sketch, the ground truth, our generated result, and the incorrectly selected strokes in the 3D sketch that led to the failure. Differences between our result and the ground truth are highlighted for clarity. The most common failures involve misclassification of small features such as fillets or chamfers, as seen in the first row.

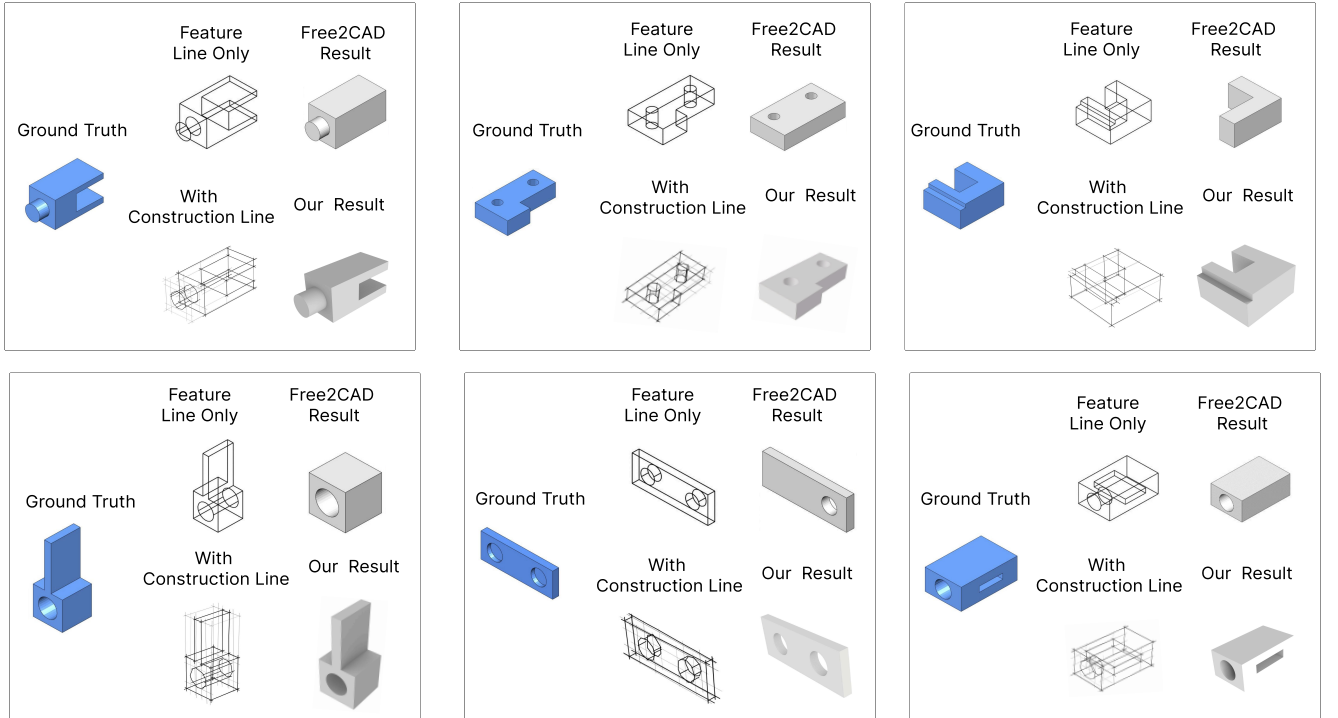


Figure 17: We present six examples from Free2CAD [LPBM22]. In each box, the top row shows the ground truth shape, the input sketch for Free2CAD, and the result generated by their method. The bottom row shows our redrawn sketch with construction lines and the corresponding result produced by our system. Original figures copied from Free2CAD.

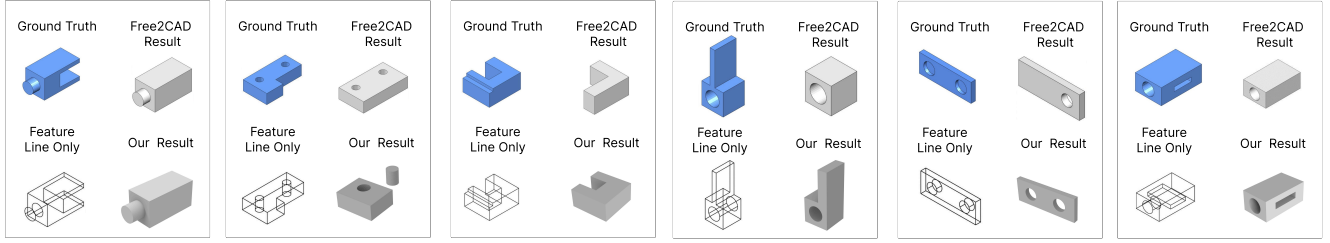


Figure 18: We present the same six examples from Free2CAD, but provide our system with sketches containing only feature lines. Under this setting, our system fails on four out of six cases, succeeding only on the two rightmost examples.

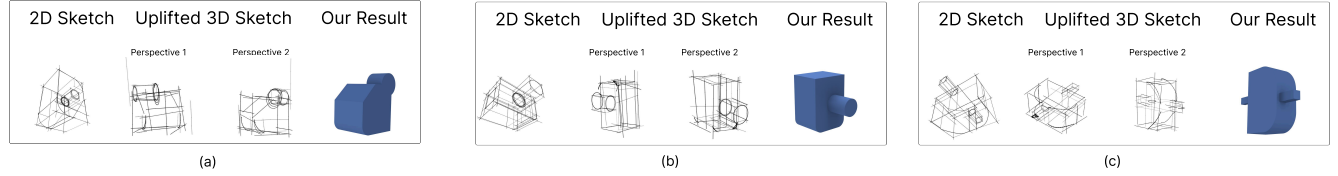


Figure 19: We selected 2D sketches from a previous work [HLMB22], and then lift them back to 3D space using a previous method [HGSB22]. We show the resulting shapes. Although the lifting approach may introduce minor issues—as seen in (c), where the circle is distorted during the uplift process—our system can still make for valid interpretations based on the 3D sketch.

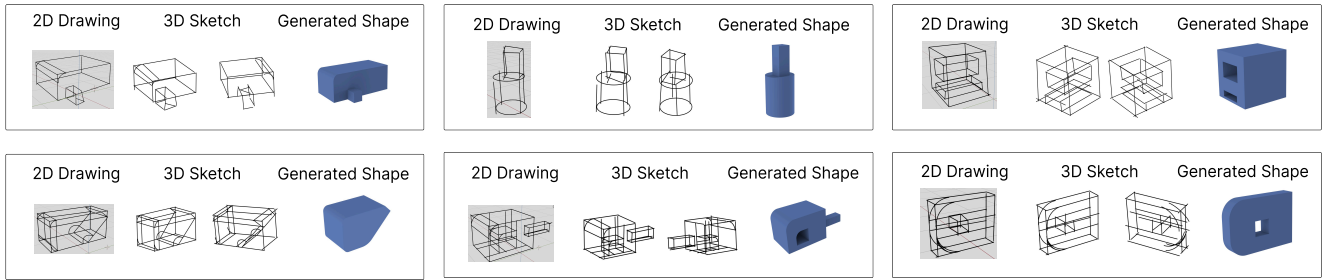


Figure 20: We invited three students with limited CAD design experience and one student proficient in CAD design to use our system. The first row presents results from a non-proficient student, while the second row shows the work of the proficient student designer.

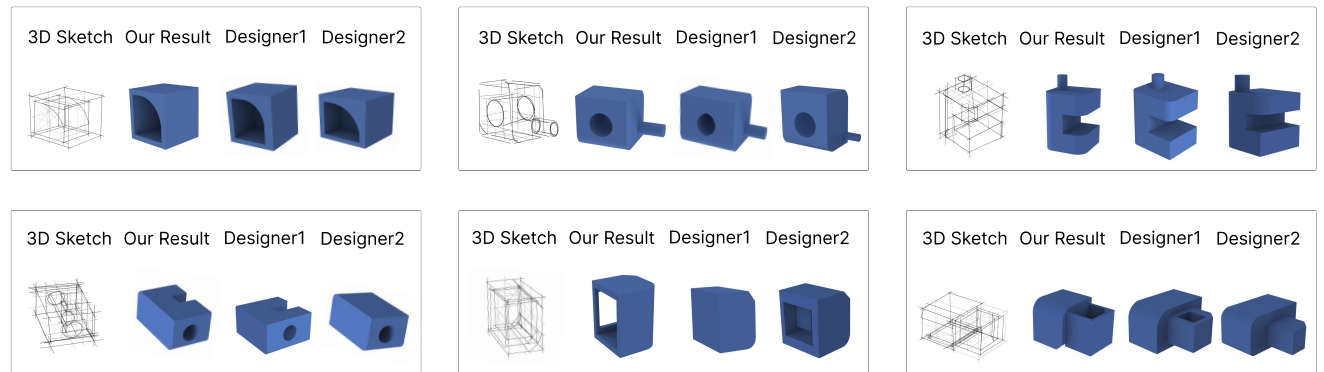


Figure 21: Comparison between reconstructions by student designers and our method. Each student was given six 3D sketches and asked to recreate the corresponding B-rep shapes. The designers performed well on simpler 3D sketches (first row), but encountered difficulties with more complex ones (second row), where many lines appear cluttered especially for 3D sketches with multiple subtractions. In contrast, our method can still handle these cases.

References

- [BBS08] BAE, SEOK-HYUNG, BALAKRISHNAN, RAVIN, and SINGH, KARAN. “ILoveSketch: as-natural-as-possible sketching system for creating 3d curve models”. *Proc. ACM Symposium on User Interface Software and Technology (UIST)*. 2008, 151–160. DOI: [10.1145/1449715.1449740](https://doi.org/10.1145/1449715.1449740) 3.
- [CF25] CHEREDDY, SATHVIK and FEMIANI, JOHN. “SketchDNN: Joint Continuous-Discrete Diffusion for CAD Sketch Generation”. *Proceedings of the 42nd International Conference on Machine Learning (ICML)*. 17 pages, 63 figures. PMLR, 2025, 1–17. DOI: [10.48550/arXiv.2507.11579](https://doi.org/10.48550/arXiv.2507.11579). URL: <https://arxiv.org/abs/2507.11579>.
- [CLS19] CHEN, XINYUN, LIU, CHANG, and SONG, DAWN. “Execution-Guided Neural Program Synthesis”. *ICLR* (2019). Presented at ICLR 2019 3.
- [CRN*22] COLLIGAN, ANDREW R., ROBINSON, TREVOR T., NOLAN, DECLAN C., et al. “Hierarchical CADNet: Learning from B-Reps for Machining Feature Recognition”. *Computer-Aided Design* 147 (C June 2022). DOI: [10.1016/j.cad.2022.103226](https://doi.org/10.1016/j.cad.2022.103226) 3.
- [CSMS13] CORDIER, FREDERIC, SEO, HYEON, MELKEMI, MAHMOUD, and SAPIDIS, NICKOLAS S. “Inferring Mirror Symmetric 3D Shapes from Sketches”. *Computer Aided Design* 45.2 (2013) 3.
- [DIP*18] DU, TAO, INALA, JEEVANA PRIYA, PU, YEWEN, et al. “InverseCSG: Automatic Conversion of 3D Models to CSG Trees”. *ACM Transactions on Graphics (Proc. SIGGRAPH Asia)* 37.6 (2018) 3.
- [ENP*19] ELLIS, KEVIN, NYE, MAXWELL, PU, YEWEN, et al. “Write, Execute, Assess: Program Synthesis with a REPL”. *ICML* (June 2019). DOI: [10.48550/arXiv.1906.04604](https://doi.org/10.48550/arXiv.1906.04604) 3.
- [ERST18] ELLIS, KEVIN, RITCHIE, DANIEL, SOLAR-LEZAMA, ARMANDO, and TENENBAUM, JOSHUA B. “Learning to Infer Graphics Programs from Hand-Drawn Images”. *NeurIPS* (July 2018). DOI: [10.48550/arXiv.1707.09627](https://doi.org/10.48550/arXiv.1707.09627) 3.
- [EWN*21] ELLIS, KEVIN, WONG, CATHERINE, NYE, MAXWELL, et al. “DreamCoder: Bootstrapping Inductive Program Synthesis with Wake-Sleep Library Learning”. *Proceedings of the ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward!)* ACM, 2021, 835–850. DOI: [10.1145/3486607.3486750](https://doi.org/10.1145/3486607.3486750). URL: <https://doi.org/10.1145/3486607.3486750> 11.
- [GHL*20] GRYADITSKAYA, YULIA, HÄHNLEIN, FELIX, LIU, CHENXI, et al. “Lifting Freehand Concept Sketches into 3D”. *TOG* 39.6 (Nov. 2020). DOI: [10.1145/3414685.3417851](https://doi.org/10.1145/3414685.3417851) 2, 3.
- [GLP*22] GUO, HAOXIANG, LIU, SHILIN, PAN, HAO, et al. “ComplexGen: CAD Reconstruction by B-Rep Chain Complex Generation”. *ACM Transactions on Graphics (SIGGRAPH)* 41.4 (2022) 3, 6.
- [GSH*19] GRYADITSKAYA, YULIA, SYPESTEYN, MARK, HOFTIJZER, JAN WILLEM, et al. “OpenSketch: A Richly-Annotated Dataset of Product Design Sketches”. *ACM Transactions on Graphics (Proc. SIGGRAPH Asia)* (2019) 2, 7.
- [GXL13] GAO, SHENG, XU, XUN, and LIN, CHUAN. “Topology Reconstruction for B-Rep Modeling from 3D Mesh in Reverse Engineering Applications”. *Computer-Aided Design* 45.2 (2013), 496–507. DOI: [10.1016/j.cad.2012.10.010](https://doi.org/10.1016/j.cad.2012.10.010). URL: https://www.researchgate.net/publication/258712788_Topology_Reconstruction_for_B-Rep_Modeling_from_3D_Mesh_in_Reverse_Engineering_Applications 6.
- [Hen12] HENRY, KEVIN. *Drawing for product designers*. Laurence King Publishing, 2012 2.
- [HGSB22] HÄHNLEIN, FELIX, GRYADITSKAYA, YULIA, SHEFFER, ALLA, and BOUSSEAU, ADRIEN. “Symmetry-driven 3D Reconstruction from Concept Sketches”. *SIGGRAPH* (July 2022). DOI: [10.1145/3528233.3530723](https://doi.org/10.1145/3528233.3530723) 2, 3, 10, 14, 21.
- [HKYM16] HUANG, HAIBIN, KALOGERAKIS, EVANGELOS, YUMER, ERSIN, and MECH, RADMIR. “Shape Synthesis from Sketches via Procedural Models and Convolutional Networks”. *IEEE Transactions on Visualization and Computer Graphics (TVCG)* 22.10 (2016), 1 3.
- [HLMB22] HÄHNLEIN, FELIX, LI, CHANGJIAN, MITRA, NILOY J., and BOUSSEAU, ADRIEN. “CAD2Sketch: Generating Concept Sketches from CAD Sequences”. *ACM Transactions on Graphics (TOG)* 41 (6 Nov. 2022), 1–18. DOI: [10.1145/3550454.3555488](https://doi.org/10.1145/3550454.3555488) 2, 7, 10, 14.
- [JGMR23] JONES, R. KENNY, GUERRERO, PAUL, MITRA, NILOY J., and RITCHIE, DANIEL. “ShapeCoder: Discovering Abstractions for Visual Programs from Unstructured Primitives”. *arXiv preprint arXiv:2305.05661* (2023). Presented at SIGGRAPH 2023. DOI: [10.48550/arXiv.2305.05661](https://doi.org/10.48550/arXiv.2305.05661). URL: <https://arxiv.org/abs/2305.05661> 11.
- [JHC*21] JONES, BENJAMIN, HILDRETH, DALTON, CHEN, DUOWEN, et al. “AutoMate: A Dataset and Learning Approach for Automatic Mating of CAD Assemblies”. *ACM Transactions on Graphics (TOG)* 40 (6 Dec. 2021), 1–18. DOI: [10.1145/3478513.3480562](https://doi.org/10.1145/3478513.3480562) 3.
- [JNK*23] JONES, BENJAMIN, NOECKEL, JAMES, KODNONGBUA, MILIN, et al. “B-rep Matching for Collaborating Across CAD Systems”. *ACM Transactions on Graphics* 42 (4 Aug. 2023). DOI: [10.1145/3592125](https://doi.org/10.1145/3592125) 3.
- [JWR22] JONES, R. KENNY, WALKER, HOMER, and RITCHIE, DANIEL. “PLAD: Learning to Infer Shape Programs with Pseudo-Labels and Approximate Distributions”. *arXiv preprint arXiv:2011.13045* (2022). Presented at CVPR 2022. DOI: [10.48550/arXiv.2011.13045](https://doi.org/10.48550/arXiv.2011.13045). URL: <https://arxiv.org/abs/2011.13045> 11.
- [KMJ*19] KOCH, SEBASTIAN, MATVEEV, ALBERT, JIANG, ZHONGSHI, et al. “ABC: A Big CAD Model Dataset for Geometric Deep Learning”. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2019, 9601–9611. DOI: [10.1109/CVPR.2019.00983](https://doi.org/10.1109/CVPR.2019.00983) 7.
- [KMP*18] KALYAN, ASHWIN, MOHTA, ABHISHEK, POLOZOV, OLEKSANDR, et al. “Neural-Guided Deductive Search for Real-Time Program Synthesis from Examples”. *arXiv* (Apr. 2018). Published in ICLR 2018, International Conference on Learning Representations. DOI: [10.48550/arXiv.1804.01186](https://doi.org/10.48550/arXiv.1804.01186) 3.
- [KSA23] KUZNETSOV, PAVEL, SPITSYN, ALEXEY, and ARUTYUNOV, ROMAN. “Simplification of 3D CAD Model in Voxel Form for Mechanical Parts Using a GAN-Based Network”. *Computer-Aided Design* 162 (2023). DOI: [10.1016/j.cad.2023.103461](https://doi.org/10.1016/j.cad.2023.103461). URL: <https://www.sciencedirect.com/science/article/pii/S0010448523001094> 6.
- [LB25] LIU, CHENXI and BESSMELTSEV, MIKHAIL. “State-of-the-art Report in Sketch Processing”. *Computer Graphics Forum* (2025) 3.
- [LCLT08] LIU, JIANZHANG, CAO, LIANGLIANG, LI, ZHENGUO, and TANG, XIAOOU. “Plane-based optimization for 3D object reconstruction from single line drawings”. *IEEE Transaction on Pattern Analysis Machine Intelligence* 30.2 (2008), 315–327 3.
- [LCP*24] LIU, YILIN, CHEN, JIALE, PAN, SHANSHAN, et al. “Split-and-Fit: Learning B-Reps via Structure-Aware Voronoi Partitioning”. *ACM Transactions on Graphics (TOG)* 43.4 (2024), 108:1–108:13. DOI: [10.1145/3658155](https://doi.org/10.1145/3658155). URL: <https://doi.org/10.1145/3658155> 6.
- [LGG*17] LIN, TSUNG-YI, GOYAL, PRIYA, GIRSHICK, ROSS, et al. “Focal Loss for Dense Object Detection”. *arXiv preprint arXiv:1708.02002* (2017). Presented at ICCV 2017. DOI: [10.48550/arXiv.1708.02002](https://doi.org/10.48550/arXiv.1708.02002). URL: <https://arxiv.org/abs/1708.02002> 5.
- [LOWS23] LIU, YUJIA, OBUKHOV, ANTON, WEGNER, JAN DIRK, and SCHINDLER, KONRAD. “Point2CAD: Reverse Engineering CAD Models from 3D Point Clouds”. *CVPR* (Dec. 2023). DOI: [10.48550/arXiv.2312.04962](https://doi.org/10.48550/arXiv.2312.04962) 3.
- [LPBM20] LI, CHANGJIAN, PAN, HAO, BOUSSEAU, ADRIEN, and MITRA, NILOY J. “Sketch2CAD: Sequential CAD Modeling by Sketching in Context”. *TOG* 39.6 (Nov. 2020). DOI: [10.1145/3414685.3417807](https://doi.org/10.1145/3414685.3417807) 2, 3.
- [LPBM22] LI, CHANGJIAN, PAN, HAO, BOUSSEAU, ADRIEN, and MITRA, NILOY J. “Free2CAD: Parsing Freehand Drawings into CAD Commands”. *TOG* 41.4 (July 2022). DOI: [10.1145/3528223.3530133](https://doi.org/10.1145/3528223.3530133) 2–4, 10, 13, 21.

- [LS96] LIPSON, H and SHPITALNI, M. "Optimization-based reconstruction of a 3D object from a single freehand line drawing". *Computer-Aided Design* 28.8 (1996) 3.
- [LWJ*22] LAMBOURNE, JOSEPH GEORGE, WILLIS, KARL, JAYARAMAN, PRADEEP KUMAR, et al. "Reconstructing editable prismatic CAD from rounded voxel models". *SIGGRAPH Asia Conference Papers*. 2022 3.
- [NGG*16] NISHIDA, GEN, GARCIA-DORADO, IGNACIO, G. ALIAGA, DANIEL, et al. "Interactive Sketching of Urban Procedural Models". *ACM Transactions on Graphics (Proc. SIGGRAPH)* (2016) 3.
- [PCV16] PLUMED, RAQUEL, COMPANY, PEDRO, and VARLEY, PETER A.C. "Detecting mirror symmetry in single-view wireframe sketches of polyhedral shapes". *Computers & Graphics* 59 (2016), 1–12 3.
- [PMKB23] PUHACHOV, IVAN, MARTENS, CEDRIC, KRY, PAUL G., and BESSMELTSEV, MIKHAIL. "Reconstruction of Machine-Made Shapes from Bitmap Sketches". *ACM Transactions on Graphics (Proc. SIGGRAPH Asia)* 42.6 (2023) 3.
- [RDM*24] RUKHOVICH, DANILA, DUPONT, ELONA, MALLIS, DIMITRIOS, et al. "CAD-Recode: Reverse Engineering CAD Code from Point Clouds". *arXiv preprint arXiv:2412.14042* (2024). DOI: [10.48550/arXiv.2412.14042](https://doi.org/10.48550/arXiv.2412.14042). arXiv: [2412.14042](https://arxiv.org/abs/2412.14042) [cs.CV] 3.
- [RGJ*23] RITCHIE, DANIEL, GUERRERO, PAUL, JONES, R. KENNY, et al. "Neurosymbolic Models for Computer Graphics". *Computer Graphics Forum* 42.2 (2023), 545–568 3.
- [SGL*18] SHARMA, GOPAL, GOYAL, RISHABH, LIU, DIFAN, et al. "CS-GNet: Neural Shape Parser for Constructive Solid Geometry". *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2018 3.
- [SHM*16] SILVER, DAVID, HUANG, AJA, MADDISON, CHRIS J, et al. "Mastering the game of Go with deep neural networks and tree search". *Nature* 529.7587 (2016), 484–489. DOI: [10.1038/nature16961](https://doi.org/10.1038/nature16961). URL: <https://www.nature.com/articles/nature16961>.
- [SIJ*07] SCHMIDT, RYAN, ISENBERG, TOBIAS, JEPPE, PAULINE, et al. "Sketching, scaffolding, and inking: A visual history for interactive 3D modeling". *Proceedings of the 5th International Symposium on Non-Photorealistic Animation and Rendering (NPAR '07)*. ACM, 2007, 23–32. DOI: [10.1145/1274871.1274875](https://doi.org/10.1145/1274871.1274875) 2.
- [SKSK09] SCHMIDT, RYAN, KHAN, AZAM, SINGH, KARAN, and KURTENBACH, GORD. "Analytic drawing of 3D scaffolds". *ACM transactions on graphics (Proc. SIGGRAPH Asia)*. Vol. 28. 5. 2009 2, 3.
- [SLK*20] SHARMA, GOPAL, LIU, DIFAN, KALOGERAKIS, EVANGELOS, et al. "ParSeNet: A Parametric Surface Fitting Network for 3D Point Clouds". *Proc. European Conference on Computer Vision (ECCV)*. 2020 3.
- [SLX*25] SUN, YUE, LI, JITUO, XU, ZIQIN, et al. "Sketch2Seq: Reconstruct CAD Models From Feature-Based Sketch Segmentation". *IEEE Transactions on Visualization and Computer Graphics* 31.10 (Oct. 2025), 8214–8230. DOI: [10.1109/TVCG.2025.3566544](https://doi.org/10.1109/TVCG.2025.3566544) 2–4, 21.
- [SS14] SADRI, BARDIA and SINGH, KARAN. "Flow-complex-based shape reconstruction from 3D curves". *ACM Transactions on Graphics* 33.2 (2014), 1–15. DOI: [10.1145/2560328](https://doi.org/10.1145/2560328) 3.
- [SSS*17] SILVER, DAVID, SCHRITTWIESER, JULIAN, SIMONYAN, KAREN, et al. "Mastering the game of Go without human knowledge". *Nature* 550.7676 (2017), 354–359. DOI: [10.1038/nature24270](https://doi.org/10.1038/nature24270). URL: <https://www.nature.com/articles/nature24270>.
- [Sto08] STORER, IAN. "Reflecting on professional practice : capturing an industrial designer's expertise to support the development of the sketching capabilities of novices". *Design and Technology Education: An International Journal* 10.1 (May 2008), 54–72 2.
- [TLS*19] TIAN, YONGLONG, LUO, ANDREW, SUN, XINGYUAN, et al. "Learning to Infer and Execute 3D Shape Programs". *arXiv* (Jan. 2019). Presented at ICLR 2019. DOI: [10.48550/arXiv.1901.02875](https://doi.org/10.48550/arXiv.1901.02875) 3.
- [UCS*22] UY, MIKAELA ANGELINA, CHANG, YEN-YU, SUNG, MIN-HYUK, et al. "Point2Cyl: Reverse Engineering 3D Objects from Point Clouds to Extrusion Cylinders". *CVPR* (June 2022). DOI: [10.48550/arXiv.2112.09329](https://doi.org/10.48550/arXiv.2112.09329) 6.
- [WB25] WEI, JIAYI and BOUSSEAU, ADRIEN, eds. *A Blender Add-on for 3D Concept Sketching*. ACM/EG Expressive Symposium - Posters and Demos. 2025. URL: <http://www-sop.inria.fr/reves/Basilic/2025/WB25> 2, 10, 21.
- [WJC*22] WILLIS, KARL D.D., JAYARAMAN, PRADEEP KUMAR, CHU, HANG, et al. "JoinABLE: Learning Bottom-up Assembly of Parametric CAD Joints". *CVPR* (Apr. 2022). DOI: [10.48550/arXiv.2111.12772](https://doi.org/10.48550/arXiv.2111.12772) 3.
- [WPL*21] WILLIS, KARL D.D., PU, YEWEN, LUO, JIELIANG, et al. "Fusion 360 Gallery: A Dataset and Environment for Programmatic CAD Construction from Human Design Sequences". *SIGGRAPH* (Aug. 2021). DOI: [10.48550/arXiv.2010.02392](https://doi.org/10.48550/arXiv.2010.02392) 7.
- [WXW18] WU, Q., XU, K., and WANG, J. "Constructing 3D CSG Models from 3D Raw Point Clouds". *Computer Graphics Forum* 37.5 (2018) 3.
- [WXZ21] WU, RUNDI, XIAO, CHANG, and ZHENG, CHANGXI. "DeepCAD: A Deep Generative Network for Computer-Aided Design Models". *ICCV* (Oct. 2021). DOI: [10.48550/arXiv.2105.09492](https://doi.org/10.48550/arXiv.2105.09492) 3, 7.
- [WZW*24] WANG, HANXIAO, ZHAO, MINGYANG, WANG, YIQUN, et al. "VQ-CAD: Computer-Aided Design model generation with vector quantized diffusion". *Computer Aided Geometric Design* 111 (2024), 102327. DOI: [10.1016/j.cagd.2024.102327](https://doi.org/10.1016/j.cagd.2024.102327). URL: <https://www.sciencedirect.com/science/article/pii/S016783962400061X> 3.
- [XCS*14] XU, BAOXUAN, CHANG, WILLIAM, SHEFFER, ALLA, et al. "True2Form: 3D Curve Networks from 2D Sketches via Selective Regularization". *ACM Transactions on Graphics (SIGGRAPH 2014 Papers)* 33 (4 2014). DOI: [10.1145/2601097.2601204](https://doi.org/10.1145/2601097.2601204) 3.
- [XPC*21] XU, XIANGHAO, PENG, WENZHE, CHENG, CHIN-YI, et al. "Inferring CAD Modeling Sequences Using Zone Graphs". *CVPR* (Apr. 2021). DOI: [10.48550/arXiv.2104.03900](https://doi.org/10.48550/arXiv.2104.03900) 3.
- [YDSG21] YU, XUE, DiVERDI, STEPHEN, SHARMA, AKSHAY, and GINGOLD, YOTAM. "ScaffoldSketch: Accurate Industrial Design Drawing in VR". *Proceedings of ACM Symposium on User Interface Software and Technology*. UIST. 2021 2, 3.
- [YLT13] YANG, LINJIE, LIU, JIANZHANG, and TANG, XIAOOU. "Complex 3d general object reconstruction from line drawings". *IEEE Int. Conference on Computer Vision*. 2013 3.
- [YZF*21] YANG, LUMIN, ZHUANG, JIAJIE, FU, HONGBO, et al. "SketchGNN: Semantic Sketch Segmentation with Graph Neural Networks". *ACM Transactions on Graphics* 40.3 (2021) 4.
- [ZHFL23] ZONG, ZHIHAO, HE, FAZHI, FAN, RUBIN, and LIU, YUXIN. "P2CADNet: An End-to-End Reconstruction Network for Parametric 3D CAD Model from Point Clouds". *CoRR* abs/2310.02638 (2023). URL: <https://arxiv.org/abs/2310.02638> 6.

Appendix A: Graph Building PseudoCode

We construct graph nodes using the algorithms described in Appendix B and graph edges using those in Appendix C. This section presents pseudocode for the complete graph construction pipeline, which consists of two phases: an initialization phase (Algorithm 1) and an update phase during program execution (Algorithm 2). After each newly generated CAD operation, the graph is updated by modifying only the binary usage flags of stroke and loop nodes, indicating whether each node is already represented in the current B-rep.

Algorithm 1 Graph Initialization Pipeline

```

1: Fit stroke types and parametric representations
2: Compute stroke nodes
3: Compute pairwise stroke intersections
4: Compute loop nodes from stroke groups
5: Compute all graph edges
6: for each stroke node do
7:   Initialize usage flag to 0
8: end for
9: for each loop node do
10:   Initialize usage flag to 0
11: end for

```

Algorithm 2 Graph Update During Program Execution

```

1: for each B-rep edge do
2:   for each stroke node do
3:     if the stroke equals or is contained by the B-rep edge
       then
4:       Mark the stroke node as used (flag  $\leftarrow$  1)
5:     end if
6:   end for
7: end for
8: for each loop node do
9:   if all connected stroke nodes are marked as used then
10:    Mark the loop node as used (flag  $\leftarrow$  1)
11:   end if
12: end for

```

Appendix B: Graph Node Feature Representation

We show the details of our graph node features in Figure 22. Both stroke nodes and loop nodes contain 12 values.

For stroke nodes, the features include parametric information, opacity, circular attributes, stroke type encoding, and a binary label. There are five stroke types: straight lines, circular arcs, full circles, ellipses, and free-form curves, as shown in Table 6.

Table 6: Node features for stroke nodes. Each stroke node has 12 values, including parametric and semantic features. e.g in the form $(x1, y1, z1, x2, y2, z2, o, -, -, l, b)$

Stroke Type	Parametric Function	Opacity	Circular Features	Stroke Type	Binary Label
Straight Line	Start and End points	Yes	/	1	0 or 1
Circular Arc	Start and End points	Yes	Center	2	0 or 1
Full Circle	Center and Normal	Yes	Radius + [0,0]	3	0 or 1
Ellipse	Center1 and Center2	Yes	Radius1, Radius2+ [0]	4	0 or 1
Free-form Curve	Start and End points	Yes	Sampled Point	5	0 or 1

For loop detection, we consider only straight lines and arcs. A circular or elliptical stroke is automatically treated as a single loop. We first detect intersections between strokes using the same stroke-to-stroke intersection criteria described in Appendix C. We then identify candidate loops by searching for groups of 3, 4, 5, or at most 6 strokes in which all strokes are mutually connected and each vertex is traversed exactly twice. Next, to verify that all strokes in a candidate loop lie on a common plane, we estimate plane normals from pairs of intersecting strokes within the group. For each stroke s , we define a direction vector $\mathbf{d}_s \in R^3$ using its start and end

points; for arcs, $\mathbf{d}_s$ is defined also as the chord direction between the arc endpoints. For every pair of intersecting strokes (s_i, s_j) , we compute a candidate plane normal as $\mathbf{n}_{ij} = (\mathbf{d}_{s_i} \times \mathbf{d}_{s_j}) / \|\mathbf{d}_{s_i} \times \mathbf{d}_{s_j}\|$. This yields a set of normal estimates $\{\mathbf{n}_{ij}\}$. The strokes are considered coplanar if the angular deviation between all candidate normals satisfies $\arccos(|\mathbf{n}_{ij} \cdot \mathbf{n}_{kl}|) < 10^\circ$ for all pairs (i, j) and (k, l) ; otherwise, the group is discarded.

For loop nodes, the first 11 values are padding, and the final value is a binary label indicating whether the loop is used. We do not assign additional features to loop nodes, as all necessary information can be derived from the strokes that form them.

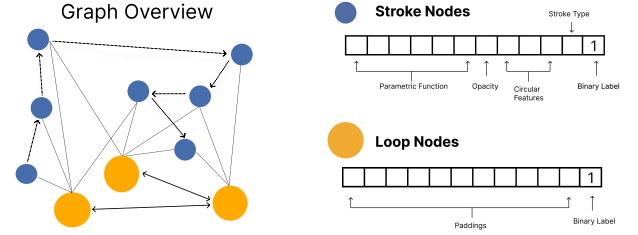


Figure 22: Overview of graph node features. We represent each stroke node using 12 values that include parametric geometry, opacity, circular characteristics, stroke type encoding, and a binary label. Loop nodes only contain a binary label in the final feature slot, with the remaining dimensions zero-padded.

Appendix C: Graph Edge Building

We construct the graph using the following types of edges. Since the input sketches are noisy, all geometric relationships are detected using tolerance thresholds, as detailed below.

- **Stroke-to-Stroke Edges.** These edges capture intersections between strokes in the 3D sketch. For straight lines, arcs, and free-form curves (represented by their endpoints), two strokes are considered intersecting if the distance between their closest endpoints is less than $0.1 \times \min(\ell_1, \ell_2)$, where ℓ_1 and ℓ_2 denote the lengths of the two strokes. For intersections involving circles or ellipses (represented by a center and radius) and strokes with endpoints, a stroke is considered intersecting a circle or ellipse if $|d - r| < 0.1 \times r$, where d is the distance from the closest endpoint to the center and r is the radius.
- **Loop-to-Loop Edges.** These edges capture intersection, containment, and perpendicular relationships between loops. Two loops are considered intersecting if they share at least one stroke. For containment, we first compute the planar area of each loop. A loop B is considered contained within loop A if all points of B lie inside the area enclosed by A or on the boundary of A . For perpendicular relationships, the two loops must share at least one stroke. Since loops may not lie on a perfect plane, we estimate a loop normal by performing PCA on the set of 3D points belonging to the loop and taking the eigenvector corresponding to the smallest eigenvalue as the normal direction. Two loops are considered perpendicular if the angle between their estimated normals (average of all normals computed in Appendix B for loop normals) deviates from 90° by less than 10° .

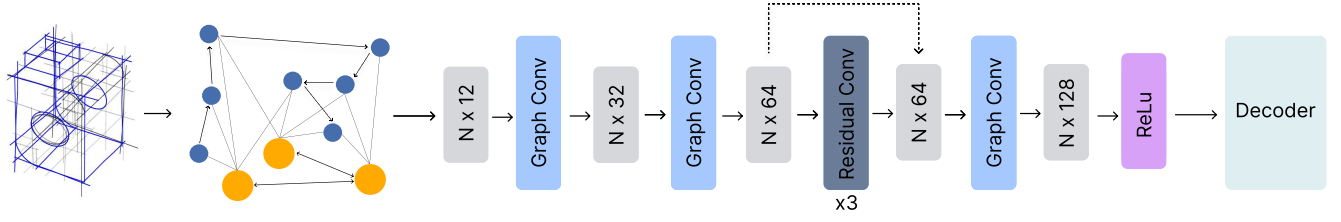


Figure 23: Overview of the graph encoder architecture. The input is a heterogeneous graph G_t , where stroke and loop nodes are initialized with 12-dimensional features. The encoder applies graph convolutions to expand features to 128 dimensions through stacked layers and residual blocks, followed by a ReLU activation before passing to the decoder for task-specific predictions.

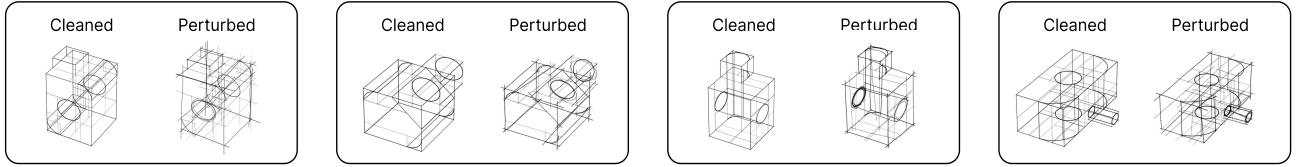


Figure 24: 3D Sketch perturbation to simulate human sketching. We show examples of clean and perturbed 3D Sketch. Perturbations are designed to emulate natural drawing variations such as jitter, overdrawing, stroke duplication, and deletion.

i.e., $\arccos(|\mathbf{n}_1 \cdot \mathbf{n}_2|) < 10^\circ$. The method for computing the loop normal differs from that described in Appendix B, where we examine the normals on all points and determine whether the points in a loop lie on a plane.

- **Stroke-to-Loop Edges.** These edges indicate which strokes constitute a given loop and are constructed automatically during loop formation (Appendix B). If a set of strokes forms a loop, each stroke is connected to the corresponding loop node via a stroke-to-loop edge.
- **Stroke-order Edges.** These edges encode the temporal order in which strokes are drawn as directed connections from each stroke to its successor in the drawing sequence.

Appendix D: Program Format Examples

We define five types of program tokens, each corresponding to a CAD operation with a fixed parameterization:

- **Profile:** `params(plane (point1, point2, point3 [, point4]))`.
- **Extrude:** `params(start plane (point1, point2, point3, point4), end plane (point5, point6, point7, point8))`.
- **Fillet:** `params(target edge (point1, point2), amount (0.1))`.
- **Chamfer:** `params(target edge (point1, point2), amount (0.1))`.
- **Terminate:** None

We show an example program that generates the shape shown in Figure 25 (values are rounded for simplicity):

```
Operation: Profile;
params(plane (
  point(0.15,0,0),
  point(0.15,0.15,0),
  point(0,0.15,0),
  point(0.15,0.15,0)
```

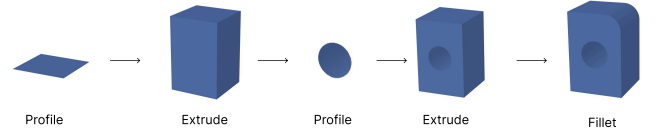


Figure 25: We show the corresponding shape constructed by the example program in Appendix D.

```
) )
```

```
Operation: Extrude;
params(
  start plane (
    point(0.15,0,0),
    point(0.15,0.15,0),
    point(0,0.15,0),
    point(0,0,0)
  ),
  end plane (
    point(0.15,0,0.25),
    point(0.15,0.15,0.25),
    point(0,0.15,0.25),
    point(0,0,0.25)
  )
)
```

```
Operation: Profile;
params(
  plane (
    circle(
      center(0.075,0,0.125),
```

```

        radius(0.03)
    )
)

Operation: Extrude;
params(
    start plane (
        circle(
            center(0.075,0,0.125),
            radius(0.03)
        )
    ),
    end plane (
        circle(
            center(0.075,0.15,0.125),
            radius(0.03)
        )
    )
)

Operation: Fillet;
params(
    Edge (
        point(0.15,0,0.25),
        point(0.15,0.15,0.25),
    ),
    Amount (0.05)
)
)

```

Appendix E: Graph Encoder Architecture

We present our graph encoder in Figure 23. All tasks in our framework utilize this shared encoder to generate latent node embeddings for both stroke and loop nodes in the graph.

Each node in the input graph is initialized with a 12-dimensional feature vector. The output of the encoder is a 128-dimensional node embedding. The encoder first applies a graph convolutional layer to project the input features into a higher-dimensional space. This is followed by three residual blocks, each consisting of two graph convolutional layers with skip connections to preserve information flow. Finally, we apply another concluding graph convolutional layer is applied, followed by a ReLU activation to produce the final node embeddings.

Appendix F: Parameter Extraction for CAD Operations

Given the strokes (or loops) associated with each operation, we extract the continuous values required to parameterize them:

- **Profile:** A loop node is selected. We first project all strokes in the loop onto the best-fitting plane. Then, we extract one unique point from each stroke, resulting in n points from n strokes. Two points are considered identical if they lie within a threshold distance of $0.2 \times \max(\text{stroke_length}_1, \text{stroke_length}_2)$. These extracted points are used to define a plane by fitting with least

squares:

$$\min_{\mathbf{n}, d} \sum_{i=1}^n (\mathbf{n}^\top \mathbf{p}_i + d)^2 \quad \|\mathbf{n}\| = 1$$

where $\mathbf{p}_i$ are the extracted points and $\mathbf{n}$ is the plane normal.

- **Extrude:** A loop node is selected as the base face. The extrusion amount is computed as the Euclidean distance between the initial and final loop planes:

$$\theta_{\text{extrude}} = \|\mathbf{loop}_{\text{end}} - \mathbf{loop}_{\text{start}}\|.$$

In practice, we also allow a fallback strategy when the selected loop plane does not match the sketch plane. In this case, we use the extruded stroke's length as the extrusion amount.

- **Fillet:** The fillet radius is directly extracted from the selected arc stroke:

$$\theta_{\text{fillet}} = r_{\text{arc}}.$$

To identify the corresponding B-rep edge, we find the edge equidistant to the two endpoints of the fillet stroke.

- **Chamfer:** The chamfer amount is approximated using the length of the selected edge:

$$\theta_{\text{chamfer}} = \frac{\|\mathbf{p}_{\text{end}} - \mathbf{p}_{\text{start}}\|}{\sqrt{2}},$$

assuming a 45° chamfer angle. Similar to the fillet case, we locate the target B-rep edge as the one equidistant from the endpoints of the chamfer stroke.

Appendix G: Extrusion Face Finding

We perform a two-round encoding–decoding process to identify the extrusion face. In the first round, the input consists of the graph and the profile face. We assign a flag to the profile loop and predict the corresponding extruded strokes. In the second round, we flag the predicted extruded strokes (while the profile loop is no longer simply marked as “used”) and then identify the extruded face. This process is illustrated in Fig. 26.

This design is motivated by two considerations. First, it is easier for the graph network to first identify the extruded strokes and then infer the extrusion face. Second, this formulation ensures that the extrusion operation is always valid, since the extrusion face is constrained to correspond to the profile face.

Appendix H: SMC Resampling Implementation

We adopt a Sequential Monte Carlo (SMC) framework to search over CAD programs. The method maintains a fixed-size set of candidate programs, referred to as *particles*. All particles are initialized from the same state, corresponding to an empty CAD program.

At each timestep t , each particle is expanded by sampling an action from the policy module, producing a new partial program. The resulting particles are then evaluated by a value function, which assigns a scalar score V_i to particle i , indicating its expected quality.

Given the particle set $\{P_t^{(i)}\}_{i=1}^N$ at timestep t and their corresponding scores $V_t^{(i)}$, we convert scores into importance weights by direct normalization, i.e., $w_t^{(i)} = V_t^{(i)} / \sum_{j=1}^N V_t^{(j)}$ (assuming $V_t^{(i)} \geq$

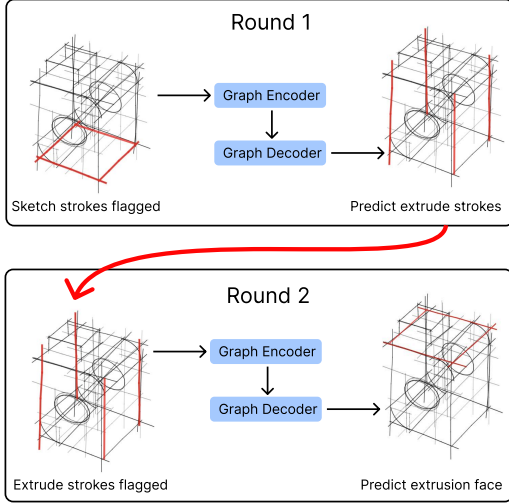


Figure 26: Two-round encoding-decoding process for extrusion: the first round predicts extruded strokes from the profile loop, and the second round identifies the resulting extrusion face using the flagged strokes.

0). We then perform multinomial resampling by drawing N indices independently with replacement from the categorical distribution defined by $\{w_t^{(i)}\}$. The resampled particle set is formed by duplicating particles corresponding to sampled indices, while particles that are not sampled are discarded.

Appendix I: Dataset Preparation using Monte Carlo Tree Search

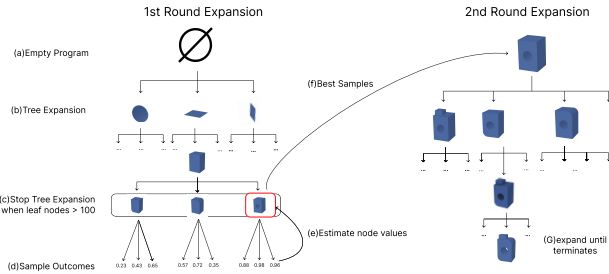


Figure 27: We adopt a two-stage tree expansion strategy. Starting from the empty program (a), we first perform a partial tree expansion (b) until reaching a budget of 100 intermediate nodes (c). We then sample program completions from each leaf node (d) to estimate node values (e). We chose the top 20 highest-scoring nodes (f), and finally fully expand these selected nodes to termination in a second stage (g).

To prepare the dataset for training our value function, we construct trees that explore all possible execution paths of the system. Since exhaustive enumeration is infeasible, we approximate this process using Monte Carlo Tree Search (MCTS), which prioritizes exploration along high-impact branches. This follows the same principle as AlphaGo-style selective expansion: the search expands

to a limited depth, estimates outcomes for remaining branches via sampling, and continues exploration only along the most promising paths while approximating the values of others.

In our implementation, enumerating the full program tree is very computationally costly. Instead, we adopt a two-stage tree expansion strategy. We first expand the program tree until it reaches 100 leaf nodes. Further expansion beyond this point becomes computationally expensive due to the rapidly increasing branching factor. To estimate the quality of these partially expanded nodes, we sample four program completions from each leaf node and use the resulting executions to estimate the value of each corresponding tree node. Based on these estimated values, we retain only the top 20 highest-scoring nodes. These selected nodes are then fully expanded until program termination to more accurately evaluate their final outcomes. For the remaining 80 nodes, we use the sampled completion results as their value estimates. This hierarchical search strategy substantially reduces the total number of explored branches while preserving the fidelity of value estimation. We show this process in Figure 27

Empirically, we observe that programs with 8 operations typically result in 300–500 tree states, while programs with 12 operations yield approximately 1200–1800 states.

Appendix J: Algorithm to Simulating Human Drawings

We propose a novel method to perturb a clean 3D sketches in order to simulate human-like drawing variations (Figure 24). The input to our system is a set of polylines, each consisting of 10 sampled points. The output has the same structure but with added perturbations.

Our perturbation process consists of two main steps. First, we perform stroke type fitting (as described in the graph construction section) to identify the type of each stroke. Second, we apply different perturbation strategies depending on the stroke type:

- **Straight Lines and Free-form Curves:** We simulate overdrawn strokes by randomly extending both the start and end points by a small fraction of the stroke length. We then perturb these endpoints in arbitrary directions, again by a fraction of the stroke length. Each intermediate point along the stroke is also randomly displaced by a small amount relative to the stroke length.
- **Arcs:** We first compute the arc’s parametric representation. We then interpolate between the arc and a straight line connecting its endpoints, randomly blending the two 3D lines. The start and end points are also perturbed in arbitrary directions based on the stroke length.
- **Full Circles and Ellipses:** We treat full circles as a special case of ellipses where the two foci coincide and the major and minor radii are equal. We perturb these by randomly displacing the center based on the radius, and randomly modifying the radius itself. Additionally, we may repeat the entire stroke with high probability to simulate overdrawing.

After applying per-stroke perturbations, we further perturb the entire 3D Sketch by randomly removing 5% of the strokes and duplicating 10% of them.

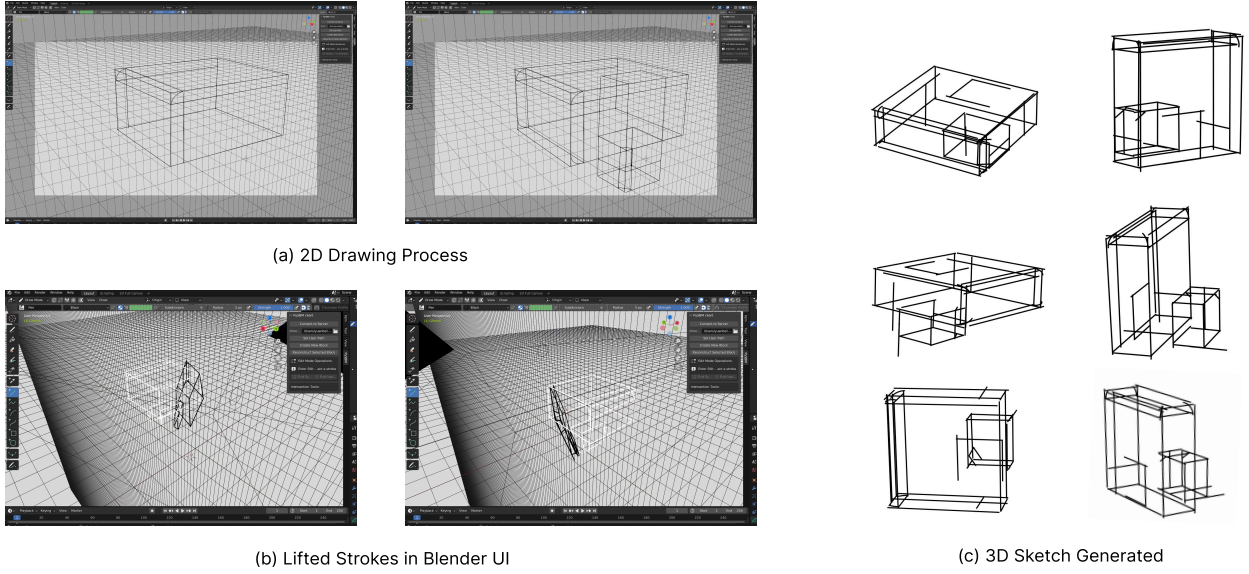


Figure 28: (a) illustrates the creation of a 2D sketch in Blender. (b) shows the corresponding lifted 3D sketch (in white) alongside the original 2D sketch (in black) within the user interface. (c) presents the final result of the lifted sketches in 3D space. Further details are provided in the original work [WB25].

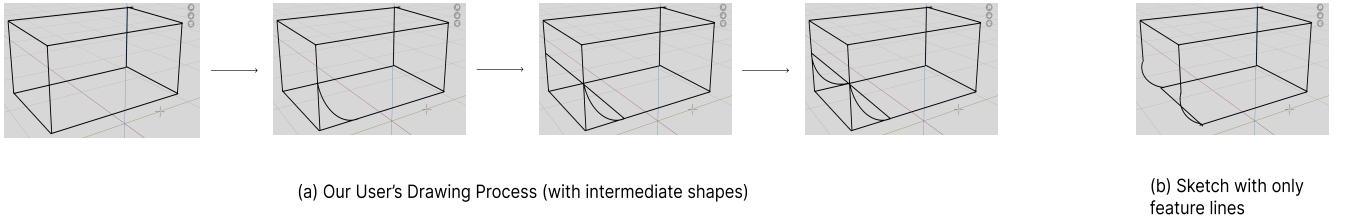


Figure 29: We present an example of a participant's drawing in (a). The participant first sketched the entire cuboid, then added a curve to indicate the fillet operation. Then the user use projection lines to connect the edges of the cuboid. These project lines help the user to maintain alignment between the two fillet curves. In contrast, (b) shows the same shape drawn with only the feature lines, which is not how people typically sketch.

Appendix K: UI Interface : Taking 2D Sketch as Input

We build on the UI system introduced in prior work [WB25], which incorporates a 2D sketch lifting algorithm [HGSB22]. We chose this system due to our familiarity with it, though our method also supports other forms of input that produce 3D sketches. The system provides a Blender add-on that allows users to create 2D drawings directly within the software, which are then automatically lifted into 3D sketches. A demonstration is shown in Figure 28, with additional details available in the original work [WB25].

Appendix L: Participants' Usage of Construction Lines

We also observed that the participants exhibited diverse sketching habits, particularly in their use of construction lines.

Intermediate Lines All participants, both proficient and non-

proficient in design, draw intermediate lines, as they found it easier to outline basic shapes first and then refine them, rather than attempting to draw the final feature lines directly (Figure 29). This observation aligns with our assumption that intermediate construction lines are frequently used in human drawings and play a crucial role in accurately recovering shapes. In contrast, prior works [LPBM22; SLX*25] does not account for such lines.

Grid Lines and Projection Lines Participants demonstrated different approaches to using additional construction lines, such as grid lines and projection lines, which do not appear in the intermediate shapes but assist in correcting perspectives. Overall, the student designer was more inclined to draw deliberate construction lines for perspective correction, whereas the other participants were less consistent. For instance, one participant never used projection or grid lines, noting that the built-in UI grid was sufficient

for maintaining alignment. Others, however, found projection lines helpful when aligning distant features. Additionally, one participant observed that drawing larger shapes with long strokes made the interface difficult to manipulate, as correcting perspective in such cases was particularly challenging. While the use of construction lines provided some assistance, the process as a whole remained cumbersome.